SIGNSIGHT: EMPOWERING ACCESSIBILITY WITH TAMIL SIGN LANGUAGE RECOGNITION USING MACHINE LEARNING

CREATE A DYNAMIC SIGN DETECTION SYSTEM USING COMPUTER VISION

Project Final Report

Mark J J (IT22328120)

B.Sc. (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

SIGNSIGHT: EMPOWERING ACCESSIBILITY WITH TAMIL SIGN LANGUAGE RECOGNITION USING MACHINE LEARNING

CREATE A DYNAMIC SIGN DETECTION SYSTEM USING COMPUTER VISION

Project Final Report

Mark J J-(IT22328120)

B.Sc. (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
Mark J J	IT22328120	mark

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor: _____ Date: _____
(Dr Prassana Sumathipala)

Signature of the co-supervisor: _____ Date: _____
(Malithi.n)

ABSTRACT

Tamil Sign Language (TSL) supports Deaf and hard-of-hearing communication in Sri Lanka, yet practical assistive tools that perform isolated sign recognition from ordinary RGB webcams, without special depth devices or studio lighting, remain scarce. This dissertation presents a SignSight research component that targets isolated gloss recognition for fifteen everyday TSL signs, using holistic landmark features and bidirectional sequence models. MediaPipe Holistic supplies 1662-dimensional landmark vectors per frame, encoding upper-body pose, facial mesh, and both hands to capture manual and many non-manual signals visible in a single view. Sequences are windowed to a fixed length: forty-five frames for a three-layer bidirectional LSTM (Bi-LSTM) classifier and thirty-two frames for a three-layer bidirectional Gated Recurrent Unit (Bi-GRU) comparator, enabling a direct contrast under slightly different temporal protocols. All features are standardised with scikit-learn's StandardScaler, then reduced with principal component analysis to two hundred fifty-six components; the scaler and PCA transform are fit only on the training split to avoid leakage, and the fitted artefacts are serialised for consistent batch training and notebook-based inference. Training applies Gaussian noise augmentation, class-weighted categorical cross-entropy to reduce bias toward frequent classes, and standard callbacks (including early stopping and learning-rate reduction on validation plateau) under a stratified eighty-five to fifteen percent train–test split. The Bi-LSTM configuration achieves approximately ninety-two point one six percent test accuracy, while the Bi-GRU reaches approximately ninety-three point four six percent, with lower parameter count. Model checkpoints and preprocessing objects are exported for demonstrated inference. Limitations include a fixed vocabulary, limited signer coverage, and the isolated sign setting; meaningful deployment would require community validation, broader data, and extension beyond isolated glosses.

Keywords: Tamil Sign Language, MediaPipe, bidirectional LSTM, bidirectional GRU, PCA

ACKNOWLEDGEMENT

The success of this research would not have been possible without the support and guidance of many individuals.

First, I express my sincere gratitude to my supervisor, Dr Prassana Sumathipala, and co-supervisor, Ms Malithi Nawarathne, for their continuous guidance, encouragement, and valuable advice throughout this research. Their expertise and constructive feedback were essential in completing the SignSight research component successfully.

I would also like to extend my thanks to all lecturers, instructors, assistant lecturers, and both academic and non-academic staff of the Sri Lanka Institute of Information Technology (SLIIT) for their support and contributions during this journey.

My heartfelt appreciation goes to my SignSight project group members for their cooperation, teamwork, and assistance during the broader project. I am grateful to everyone who supported data collection, review, and testing in relation to this work. I also thank the volunteers who contributed screen recordings that made empirical evaluation of this component possible. Responsibility for any remaining limitations in the dissertation rests with the author.

TABLE OF CONTENTS

DECLARATION 3

ABSTRACT 4

ACKNOWLEDGEMENT 5

TABLE OF CONTENTS 6

LIST OF FIGURES 15

LIST OF TABLES 16

LIST OF ABBREVIATIONS 17

LIST OF APPENDICES 17

CHAPTER 1 — INTRODUCTION 18

1.1 Background and Literature Survey 18

1.1.1 Thematic Strands in Sign Language Recognition Literature 20

1.2 Literature Review 21

1.2.1 From Hand-Crafted Features to Learned Representations 21

1.2.2 Deep Learning for Sign Language: Appearance Streams 21

1.2.3 Holistic and Multi-Cue Fusion 21

1.2.4 Sequence Modelling: RNNs, CTC, and Transformers 21

1.2.5 Evaluation Culture and Fairness 22

1.2.6 Synthesis 22

1.3 Research Gap 22

1.4 Research Problem 23

1.5 Research Objectives 24

1.6 Organisation of the Dissertation 25

1.7 Regional Context — Sign Language Technology and Inclusion in South Asia 25

1.7.1 Communication Access as a Development Theme 26

1.7.2 Low-Resource Sign Languages and Research Ethics 26
1.7.3 Technology Transfer from Global Benchmarks 26
1.7.4 Hardware Realities in Educational Institutions 26
1.7.5 Public Sector and NGO Collaboration Pathways 27
1.7.6 Intersectionality and Representation 27
1.7.7 Synthesis 27
1.8 Vocabulary Descriptions for Engineering Traceability 27
1.8.1 Beautiful 27
1.8.2 Drink 28
1.8.3 Eat 28
1.8.4 Five 28
1.8.5 Good 28
1.8.6 Hello 28
1.8.7 House 28
1.8.8 Love 28
1.8.9 Man 28
1.8.10 Mother 28
1.8.11 Run 29
1.8.12 Thank You 29
1.8.13 White 29
1.8.14 Yellow 29
1.8.15 You 29
CHAPTER 2 — METHODOLOGY 29
2.1 Overview of Research Method and Design 29
2.2 System Context and Architectural Rationale 33

2.3 Requirements Analysis 33
2.3.1 Functional Requirements 33
2.3.2 Non-Functional Requirements 34
2.4 Use Case and Behavioural Views 34
2.4.1 Use Case Scenarios 36
2.5 Dataset Design and Collection Protocol 41
2.5.1 Vocabulary and Cardinality 41
2.5.2 Storage Schema 41
2.5.3 Capture and Extraction Loop 42
2.5.4 Quality Considerations During Collection 42
2.6 Mathematical Description of the Per-Frame Feature Vector 42
2.7 From Raw Sequences to Learning Problems 43
2.8 Feature Scaling and Principal Component Analysis 43
2.8.1 Motivation 43
2.8.2 PCA Objective and Implementation 44
2.8.3 Explained Variance Reporting 44
2.9 Data Augmentation and Class Balancing 44
2.9.1 Gaussian Noise Augmentation 44
2.9.2 Class Weights 45
2.10 Recurrent Temporal Modelling 45
2.10.1 Bidirectional Recurrence 45
2.10.2 LSTM Cell Summary 45
2.10.3 GRU Cell Summary 46
2.10.4 Global Average Pooling Over Time 46
2.10.5 Multilayer Perceptron Head 46

2.11 Loss Function, Optimisation, and Callbacks 47
2.11.1 Categorical Cross-Entropy 47
2.11.2 Adam Optimiser 48
2.11.3 Callback Strategy 48
2.12 Evaluation Protocol and Metrics 48
2.12.1 Hold-Out Accuracy 48
2.12.2 Diagnostic Curves 49
2.12.3 Confusion Analysis 49
2.13 Real-Time Inference and Deployment Architecture 49
2.14 Testing and Implementation 50
2.14.1 Unit-Level Checks 50
2.14.2 Integration Tests 50
2.14.3 Qualitative Tests 50
2.15 Commercialisation Aspects of the Product 50
2.16 Risk, Ethics, and Social Impact 51
2.17 Chapter Summary 51
2.18 Algorithmic Procedures (Pseudocode-Level Detail) 52
2.18.1 Procedure COLLECT_SEQUENCE 52
2.18.2 Procedure BUILD_TENSOR_DATASET 52
2.18.3 Procedure FIT_PREPROCESSORS 53
2.18.4 Procedure TRAIN_CLASSIFIER 53
2.19 Hyperparameter Justification and Sensitivity Discussion 53
2.20 Threats to Validity and Mitigations 54
2.21 Reproducibility Checklist 54
2.22 Extended Discussion of Related Engineering Alternatives 55

2.23 Detailed Testing Matrix (Illustrative) 55
2.24 Chapter Bridge 55
2.25 Software Engineering Lifecycle and Project Governance 56
2.26 Standards, Privacy, and Data Protection Considerations 56
2.27 Detailed Description of MediaPipe Holistic Processing 56
2.28 Alternative Preprocessing Paths Considered 57
2.29 Integration Testing and Continuous Verification 57
2.30 Documentation and Knowledge Transfer for Group Continuity 57
2.31 Limitations of the Methodology Chapter's Scope 57
2.32 Closing Remarks on Methodology Rigor 58
2.33 Mathematical Foundations of Recurrent Modelling 59
2.33.1 Notation 59
2.33.2 LSTM Update Equations (Forward Direction) 59
2.33.3 GRU Update Equations (Forward Direction) 60
2.33.4 Bidirectional Composition 60
2.33.5 Dropout on Recurrent Connections 60
2.33.6 Global Average Pooling 60
2.33.7 Softmax Classifier and Cross-Entropy 61
2.33.8 Optimisation Landscape Intuition 61
2.34 Principal Component Analysis: Algebraic Summary 61
2.35 Stratified Sampling and Class Proportions 61
2.36 Confidence Scores and Calibration (Advanced Note) 62
2.37 Computational Complexity Sketch 62
2.38 Relation to Continuous Recognition Objectives 62
2.39 Chapter Addendum — Mathematical Summary 62

2.40 Implementation Walkthrough Aligned with Project Notebooks 62
2.40.1 Data Collection Notebook Responsibilities 63
2.40.2 Preprocessing and LSTM Training Notebook 63
2.40.3 GRU Training Notebook 63
2.40.4 Inference Notebook (GRU) 63
2.40.5 Artefact Inventory 63
2.40.6 Common Failure Modes in Student Replication 64
2.40.7 Version Control and Notebook Hygiene 64
2.40.8 Handover Checklist for SignSight Integration 64
2.41 Software Development Life Cycle (SDLC) Perspective 65
2.41.1 Requirements Traceability Matrix (Illustrative) 65
2.41.2 Configuration Management 65
2.41.3 Defect Triage During Development 65
2.41.4 Release Packaging Criteria 66
2.42 Quality Assurance (QA) Layers 66
2.43 Documentation Standards for Final Submission 66
2.44 Professional Communication of Limitations 66
2.45 Longitudinal Maintenance and Technical Debt 66
2.46 Security Considerations for Future Web Deployment 66
2.47 Performance Engineering Notes 67
2.48 Accessibility of the Development Environment Itself 67
2.49 Lessons Learned Summary 67
2.50 Transition to Empirical Results 67
2.51 Information-Theoretic Intuition for Vocabulary Size and Window Length 67
2.52 Sample Complexity Considerations 68

2.53 Dataset Statistics: Moments and Trajectory Smoothness 68
2.54 Correlation Structure Across Landmarks 68
2.55 Label Noise and Annotation Integrity 68
2.56 Imbalance Scenarios in Expanded Datasets 68
2.57 Cross-Validation Versus Single Hold-Out 68
2.58 Sign Language Modality in Computational Terms 69
2.59 Coarticulation and Isolated Sign Idealisation 69
2.60 Multimodal Fusion Beyond Landmark Concatenation 69
2.61 Reproducible Randomness Policy 69
2.62 Open Science and Artefact Sharing 69
2.63 Benchmark Hygiene 69
2.64 Pedagogical Value for Undergraduate Programmes 70
2.65 Final Methodological Reflection 70
2.66 Training Dynamics and Optimisation — Extended Treatment 70
2.66.1 Epoch-Level Versus Step-Level Perspectives 70
2.66.2 Underfitting Signatures 70
2.66.3 Overfitting Signatures 71
2.66.4 Learning Rate Warm-up and Cool-down 71
2.66.5 Gradient Clipping in Practice 71
2.66.6 Batch Normalisation Placement 71
2.66.7 Checkpoint Frequency and Disk Management 71
2.66.8 TensorBoard and Experiment Tracking 71
2.66.9 Reproducibility and Hardware Variance 71
2.66.10 Energy and Sustainability Considerations 72
2.66.11 Human Time Versus Compute Time 72

2.66.12 Post-Training Quantisation (Future) 72
2.66.13 Model Ensembling (Future) 72
2.66.14 Imitation Learning and Behaviour Cloning (Out of Scope) 72
2.66.15 Final Training Summary 72
2.67 Experimental Record-Keeping Standards 72
2.68 Supervisor Review Gates 72
2.69 Intellectual Honesty in Reporting Cherry-Picked Clips 73
2.70 Closing of Extended Training Discussion 73
CHAPTER 3 — RESULTS AND DISCUSSION 73
3.1 Results 73
3.1.1 Experimental Setting Summary 73
3.1.2 Quantitative Outcomes 73
3.1.3 Learning Curves 74
3.1.4 Confusion Structure 75
3.1.5 Comparative Bar Summary 76
3.1.6 Software Versions 77
3.2 Research Findings 77
3.3 Discussion 78
3.3.1 Interpretation of Accuracy Magnitudes 78
3.3.2 Error Modes 78
3.3.3 Ethics and Stakeholder Communication 78
3.3.4 Relation to SignSight Roadmap 79
3.4 Summary of Each Student's Contribution 79
3.5 Brief Integration of Conceptual Model 79
3.6 Extended Discussion — Error Taxonomy and Remediation Strategies 80

3.6.1 Perception-Layer Errors 80
3.6.2 Temporal Segmentation Errors 81
3.6.3 Linguistic and Lexical Ambiguity 81
3.6.4 Dataset Bias 81
3.7 Broader Impacts — Education and Inclusion 81
3.8 Comparison to Alternative Modalities (Qualitative) 81
3.9 Statistical Notes on Small Test Sets 81
3.10 Implications for Future Dataset Versioning 82
3.11 Knowledge Contributions Stated Compactly 82
3.12 Closing Bridge to Chapter 4 82
3.13 Narrative Walkthrough of a Successful Classification Episode 82
3.14 Narrative Walkthrough of a Failure Episode 82
3.15 Comparative Reading: LSTM versus GRU in This Corpus 82
3.16 Implications for Curricula and Capacity Building 83
3.17 Visualising Errors for Supervision Meetings 83
3.18 Connection to SLIIT Learning Outcomes 83
3.19 Closing Summary Before Chapter 4 83
3.20 Supplementary Results Interpretation — Parameter Efficiency 84
3.21 Supplementary Results Interpretation — Temporal Context 84
3.22 Communicating Uncertainty to Non-Experts 84
3.23 Replication Packet for Independent Auditors 84
3.24 Positioning Within Peer Undergraduate Projects 84
3.25 Final Pre-Conclusion Remark 84
3.26 Extended Results — Practical Interpretation for Educators 84
3.27 Extended Results — Practical Interpretation for Developers 85

3.28 Extended Results — Practical Interpretation for Policymakers 85

3.29 Meta-Analysis of Metric Reporting Conventions 85

3.30 Student Reflection on Interdisciplinary Learning 85

3.31 Final Paragraph Before Conclusions Chapter 85

CHAPTER 4 — CONCLUSIONS AND RECOMMENDATIONS 85

4.1 Summary of Work 85

4.2 Key Conclusions 86

4.3 Recommendations 86

4.4 Final Remarks 86

GLOSSARY 89

APPENDIX A — EXTENDED EVALUATION AND TESTING PROTOCOLS 90

A.1 Evaluation Protocol Checklist 90

A.2 Metrics Beyond Accuracy 90

A.3 Qualitative Review Sessions 91

A.4 Stress Tests 91

A.5 Regression Testing After Retraining 91

A.6 Documentation Deliverables for Examination 91

APPENDIX B — GLOSS INVENTORY TABLE 91

APPENDIX C — FIGURE GENERATION REPRODUCIBILITY 92

LIST OF FIGURES

Figure 1.1 Conceptual model — SignSight accessibility context .....

Figure 2.1 System workflow .....

Figure 2.2 Data flow diagram .....

Figure 2.3 Use case diagram .....

Figure 2.4 Sequence diagram (inference) .....

Figure 2.5 Model training pipeline .....

Figure 2.6 Deployment architecture (prototype) .....

Figure 2.7 Layered system architecture .....

Figure 2.8 Recurrent classifier architecture / block structure .....

Figure 3.1 Model architecture block diagram .....

Figure 3.2 Bi-LSTM accuracy curves .....

Figure 3.3 Bi-GRU accuracy curves .....

Figure 3.4 Normalised confusion matrix .....

Figure 3.5 Hold-out accuracy comparison .....

LIST OF TABLES

Table 1.1 Research gap mapping .....

Table 2.1 Illustrative functional test cases .....

Table 2.2 Requirements traceability (excerpt) .....

Table 2.3 Use case scenario summary .....

Table 3.1 Comparative model performance .....

Table 3.2 Software stack .....

Table B.1 Gloss inventory (Appendix) .....

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
Bi-GRU	Bidirectional Gated Recurrent Unit
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
PCA	Principal Component Analysis
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SLIIT	Sri Lanka Institute of Information Technology
TSL	Tamil Sign Language

LIST OF APPENDICES

Appendix A Extended evaluation and testing protocols .....

Appendix B Gloss inventory table .....

Appendix C Figure generation reproducibility .....

CHAPTER 1 — INTRODUCTION

1.1 Background and Literature Survey

Communication accessibility is a foundational requirement for equitable participation in education, employment, healthcare, and civic life. For Deaf and hard-of-hearing communities, signed languages are not optional substitutes for speech but fully fledged linguistic systems with their own grammar, prosody, and pragmatics. In Sri Lanka, Tamil Sign Language (TSL) functions as a primary channel of expression and social connection for a significant segment of the population. National estimates of hearing impairment vary by source and methodology; regardless of the precise figure, the qualitative need is clear: individuals who sign TSL frequently encounter environments—classrooms, workplaces, service counters, and digital products—that are organised around spoken languages and auditory cues. Bridging that gap with technology is therefore both a social and an engineering challenge.

Historically, assistive communication technologies for sign languages have progressed along several trajectories. Early research emphasised instrumented gloves, electromagnetic sensors, or controlled laboratory capture because these modalities simplified the feature extraction problem: when joint angles are measured directly, the learning system can focus on temporal patterning rather than robust visual inference. While such approaches can achieve high accuracy under calibrated conditions, they impose hardware costs, maintenance burdens, and adoption friction that are poorly aligned with widespread deployment in schools and homes. Consumer-grade monocular RGB video, by contrast, is ubiquitous. Webcams integrated into laptops and mobile devices offer a practical sensing path provided that the accompanying software stack is reliable on mid-range hardware and honest about failure modes when lighting, motion blur, or occlusion degrade visual evidence.

Computer vision for human pose and hand analysis has matured substantially over the past decade. Frameworks such as Google's MediaPipe provide efficient graphs for multi-part landmark estimation, including upper-body pose, dense facial geometry, and hand skeletons, often at real-time frame rates on CPU-only machines. For sign language recognition research, this modality choice carries an important design implication: the learning problem can be reframed from "classify raw pixels" to "classify compact geometric trajectories." The latter representation tends to reduce sensitivity to clothing texture and background clutter at the cost of dependence on

landmark estimator quality. When landmarks fail—due to motion blur, selfocclusion, or low contrast—the downstream classifier receives noisy or missing inputs. A rigorous methodology must therefore document not only model architecture but also preprocessing rules that stabilise sequences and align training distributions with inference-time buffering.

Parallel to sensing advances, sequence modelling for gesture and action recognition has diversified. Convolutional neural networks operating on stacks of RGB frames remain strong baselines when large labelled video corpora exist and when computational budgets permit extensive training. Recurrent models, including Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), continue to offer interpretable temporal baselines for moderate-length sequences, especially when each time step is already represented by a moderatedimensional feature vector. Bidirectional recurrent layers aggregate information from both earlier and later time indices within a fixed window, which is particularly appropriate for isolated sign classification where the entire clip is available before a decision must be rendered. More recent transformer encoders substitute self-attention for recurrence and can excel when datasets are large enough to support parameter-heavy attention models; for student-scale corpora collected in-house, recurrent baselines often remain attractive because they train stably and produce checkpoints that are easy to deploy in notebook-first workflows.

The present research component is situated within SignSight, a broader initiative oriented toward accessibility through Tamil Sign Language technologies. The component scope is deliberately constrained to isolated gloss recognition: the system predicts one label from a short video segment drawn from a fixed vocabulary of fifteen everyday signs. This scope is narrower than conversational translation or continuous recognition, but it is nonetheless technically demanding because isolated signs still exhibit coarticulation effects, variable signing speed, and interpersonal stylistic differences. Moreover, the constrained vocabulary supports reproducible evaluation and modular integration: upstream processes can segment candidate clips, while downstream services can map predicted gloss identifiers to lesson content, textual captions, or analytics dashboards.

From a software engineering perspective, the dissertation emphasises train-serve consistency. Landmark extraction, temporal windowing, scaling, and dimensionality reduction are implemented so that the same transformations learned during offline training can be reapplied during live capture using serialised artefacts. This design choice mitigates a common failure mode in student projects:

training on one set of ad hoc normalisation rules while deploying with a different rule set, producing silently degraded accuracy. The exported bundle—neural network weights, scaler parameters, PCA projection matrix, and label vocabulary—defines an explicit integration contract for sibling modules within SignSight.

The literature on sign language recognition frequently reports results on American Sign Language (ASL) or British Sign Language (BSL) benchmarks where community resources and standard splits are more established. Tamil Sign Language, like many regional sign languages, lacks comparably large public corpora and standardised evaluation protocols. That imbalance does not diminish the value of local research; it raises the bar for methodological transparency. Claims must be proportionate to evidence, datasets must be described with sufficient detail for replication or extension, and limitations on signer diversity and environmental variation must be stated plainly. This dissertation adopts that posture throughout.

In summary, the background motivates a holistic-landmark, recurrent-sequence approach to isolated TSL gloss recognition under commodity hardware assumptions. The next subsection sharpens this motivation by reviewing thematic strands in prior work that directly inform design decisions in Chapters 2 and 3.

1.1.1 Thematic Strands in Sign Language Recognition Literature

Three strands are especially relevant. First, **appearance-based** methods treat video as a stack of images or short clips and learn spatio-temporal filters end-to-end. These methods can capture subtle textural cues but may overfit to backgrounds and clothing when datasets are small. Second, **skeleton- or landmark-based** methods reduce each frame to a structured set of coordinates. These methods align naturally with MediaPipe outputs and emphasise motion geometry rather than pixel appearance. Third, **linguistically informed** approaches emphasise non-manual markers, mouthing, and grammatical facial expressions. While the present pipeline encodes facial landmarks, it does not replace linguistic annotation; rather, it leverages partial non-manual evidence available in a holistic mesh.

Cross-cutting concerns include **signer-independence** evaluation, **continuous** signing with segmentation, and **translation** to spoken language text. This dissertation does not solve those

harder problems; instead, it contributes a reproducible isolated-sign baseline that can be extended when additional labelled data and community partnerships become available.

1.2 Literature Review

This section surveys prior research themes that motivate the design decisions in Chapters 2 and 3.

1.2.1 From Hand-Crafted Features to Learned Representations

Early gesture recognition relied on hand-crafted features—contours, optical flow histograms, and trajectory descriptors—paired with classifiers such as Hidden Markov Models (HMMs). Starner and Pentland's influential work on real-time American Sign Language recognition demonstrated the viability of video-driven HMM pipelines, but also highlighted dependence on controlled environments and limited vocabularies. As compute became cheaper, researchers shifted toward learned representations, first via shallow neural networks and later via deep convolutional and recurrent architectures.

1.2.2 Deep Learning for Sign Language: Appearance Streams

Deep CNNs excel when large labelled datasets exist. Koller et al. advanced large-scale deep sign language recognition, illustrating how curated corpora enable strong CNN baselines. For lowresource languages, however, data volume may not justify end-to-end RGB training without transfer learning or heavy augmentation. Moreover, CNN models can entangle appearance factors (clothing, background) with linguistic content unless carefully evaluated.

1.2.3 Holistic and Multi-Cue Fusion

Gesture recognition systems increasingly fuse cues: hands, arms, face, and body posture. Neverova et al. explored adaptive multi-modal gesture recognition, emphasising that different modalities contribute complementary evidence depending on context. MediaPipe Holistic operationalises a pragmatic subset of this idea for real-time applications by packaging pose, face, and hand graphs in a unified pipeline.

1.2.4 Sequence Modelling: RNNs, CTC, and Transformers

For isolated classification with aligned segments, recurrent models and softmax classifiers suffice. For continuous signing, alignment-free objectives such as Connectionist Temporal Classification (CTC) become relevant. Transformers and sign-language-specific transformer variants

demonstrate strong performance on large benchmarks when pretraining and data scaling are available. The present dissertation defers continuous recognition but positions the isolated baseline as compatible with future CTC or segmentation modules.

1.2.5 Evaluation Culture and Fairness

Benchmark leadership on ASL does not automatically translate to community benefit. Academic prototypes must be evaluated with humility: report who is represented in data, what environments were used, and what failures look like for real users. This dissertation adopts explicit limitation language to align with this norm.

1.2.6 Synthesis

The literature supports a landmark-first strategy for modest corpora, bidirectional temporal encoding for isolated clips, and honest scoping for regional sign languages. These choices motivate the architecture and evaluation protocol in later chapters.

1.3 Research Gap

Despite progress in global sign language recognition research, several gaps persist for Tamil Sign Language and for deployment-oriented student laboratories:

1. **Language-resource gap:** Many published baselines assume large ASL/BSL datasets. TSL lacks equivalent open resources, which discourages direct comparison and encourages opaque "accuracy-only" reporting without dataset description.

2. **Modality gap:** Glove- and depth-based systems achieve strong controlled-environment results but are less practical for schools and homes than monocular RGB pipelines.

3. **Train-serve gap:** Prototypes often omit explicit serialisation of preprocessing transforms, yielding integration failures when models move from notebooks to demonstration applications.

4. **Temporal-protocol gap:** Fixed-length windowing rules are frequently left implicit, making it difficult to reproduce experiments or to interpret differences between training and live inference.

5. **Comparative modelling gap:** Student projects sometimes report a single architecture without controlled comparison under identical preprocessing, obscuring whether gains arise from features or from architecture.

This research component addresses these gaps by documenting a full pipeline—from folderorganised videos through MediaPipe Holistic landmarks, standardisation and PCA, to bidirectional LSTM and GRU classifiers—with explicit window lengths (45 vs 32 frames), stratified evaluation, exported artefacts, and side-by-side comparison of recurrent cores.

Table 1.1 summarises the positioning of the proposed system relative to common baseline assumptions in the literature.

Table 1.1 — Research gap mapping (Introduction)

Issue	Typical pattern in literature	This dissertation
Target language	ASL/BSL benchmarks	TSL-oriented isolated gloss vocabulary (15 classes)
Input sensing	Specialised hardware or lab capture	Consumer RGB + MediaPipe Holistic
Feature representation	Pixels or cropped hand patches	1662-D holistic landmark vector per frame
Preprocessing transparency	Partially documented	StandardScaler + PCA(256), joblib export
Temporal model	Single architecture	Bi-LSTM (45 frames) vs Bi-GRU (32 frames)
Evaluation	Accuracy only	Stratified split, curves, confusion analysis, parameter count

1.4 Research Problem

The central research problem is to determine whether a **holistic landmark sequence** pipeline, coupled with **bidirectional recurrent** sequence classifiers, can achieve **reliable isolated gloss recognition** for a **fixed TSL-linked vocabulary** using **commodity webcams**, while maintaining **train-serve alignment** suitable for integration into the **SignSight** platform.

Formally, let a video clip be represented by a sequence of RGB frames. The perception module maps each frame to a vector in $\mathbb{R}^{1662}$. A temporal windowing function maps variable capture lengths to a fixed tensor in $\mathbb{R}^{(T\times1662)}$, which is subsequently scaled and projected to $\mathbb{R}^{(T\times256)}$. The learning problem is to estimate a classifier $f: \mathbb{R}^{(T\times256)} \rightarrow \Delta^{14}$ (15-way probability simplex) that minimises empirical risk under a stratified split while generalising to unseen clips collected under similar conditions.

The research also asks a comparative question: under identical preprocessing and head topology, how do **Bi-LSTM** and **Bi-GRU** differ in **hold-out accuracy** and **parameter efficiency** when **T** differs (45 vs 32)?

Finally, the problem includes an engineering dimension: what **minimum artefact bundle** and **interface contract** enable sibling modules to consume predictions without accidental scaler mismatch?

1.5 Research Objectives

Main objective. To design, implement, and evaluate an isolated Tamil Sign Language gloss recognition module for SignSight using MediaPipe Holistic landmarks and bidirectional recurrent networks, with documented methodology, exported preprocessing artefacts, and stratified quantitative evaluation.

Specific objectives.

1. To construct a balanced corpus of **1020** isolated sequences (**68** recordings per gloss) across **fifteen** gloss classes with consistent on-disk organisation suitable for automated loading.

2. To implement robust **1662-dimensional** landmark extraction combining pose, face, and left/right hand streams using MediaPipe Holistic.

3. To define and implement **temporal windowing** policies for **45-frame** (LSTM) and **32-frame** (GRU) pipelines, documenting alignment between collection, training, and inference notebooks.

4. To apply **feature standardisation** and **PCA-based dimensionality reduction** fitted exclusively on training data; to serialise models for reuse during live inference.

5. To train a **three-layer bidirectional LSTM** classifier with dropout-based regularisation, batch normalisation, and (in the implemented LSTM configuration) L2 weight penalties on recurrent and dense kernels.

6. To train a **three-layer bidirectional GRU** classifier mirroring hidden widths and dense head topology for controlled comparison.

7. To evaluate models using **stratified 85/15** splitting, **categorical accuracy**, **loss dynamics**, and **confusion-matrix** diagnostics; to report **parameter counts** and comparative **test accuracy**.

8. To provide a **demonstration inference** pathway (notebook-based) for **webcam** and **file upload**, including confidence-aware messaging suitable for iterative user testing.

9. To articulate **limitations** pertaining to vocabulary size, signer diversity, continuous signing, and community validation, and to outline a **future roadmap** aligned with responsible deployment.

These objectives align with the SLIIT expectation that an undergraduate dissertation advances a coherent technical point of view supported by reproducible evidence rather than anecdotal demonstration.

1.6 Organisation of the Dissertation

Chapter 2 presents methodology, including system architecture, data collection, preprocessing, model design, training, testing, commercialisation considerations, and implementation details. Chapter 3 reports results, findings, and discussion, including a summary of the student's contribution within the group context where applicable. Chapter 4 concludes with recommendations. Supporting lists, figures, and appendices follow SLIIT formatting guidance.

End of Chapter 1 narrative body (expand further in final Word assembly if pagination requires additional literature citations per supervisor feedback).

1.7 Regional Context — Sign Language Technology and Inclusion in South Asia

While Chapter 1 established the technical motivation for Tamil Sign Language (TSL) recognition, a dissertation situated in Sri Lanka also benefits from explicit discussion of **regional context** and **inclusion policy framing**. This section does not substitute for sociolinguistic expertise; rather, it

orients engineering decisions within broader patterns of accessibility infrastructure development in South Asia and comparable settings.

1.7.1 Communication Access as a Development Theme

Communication access intersects with education policy, employment nondiscrimination norms, and healthcare delivery. In many countries, legislative frameworks mandate reasonable accommodation, yet implementation gaps persist at the classroom and service-desk level. Digital tools can narrow gaps when they are affordable, maintainable, and co-designed with Deaf communities. Conversely, poorly tested prototypes can waste stakeholder time and erode trust. The SignSight initiative implicitly adopts the second posture—building incrementally with transparent limitations—by foregrounding modular recognition and reproducible evaluation.

1.7.2 Low-Resource Sign Languages and Research Ethics

"Low-resource" in natural language processing often denotes limited textual corpora; for sign languages, the scarcity extends to **video corpora**, **linguistic annotation**, and **standardised lexicons**. Tamil Sign Language, like many regional sign languages, may exhibit dialectal variation and incomplete standardisation in written gloss inventories used by engineers. When English gloss names label classes, they function as **project-internal identifiers** rather than authoritative linguistic documentation. Future collaborations with Deaf-led organisations and sign linguists should refine lexical choices and align labels with community preferences.

1.7.3 Technology Transfer from Global Benchmarks

Researchers frequently import methods validated on ASL or BSL benchmarks. Transfer is intellectually legitimate if hypotheses are re-tested under local data regimes. However, uncritical transfer risks **cultural misalignment**: signs differ not only in handshape but in grammar and non-manual marking. The present pipeline's holistic landmarks partially encode non-manual cues but do not replace expert annotation. Claims must remain scoped to **isolated gloss discrimination** on a **project vocabulary**.

1.7.4 Hardware Realities in Educational Institutions

Sri Lankan universities and schools vary in laboratory endowments. A pipeline that trains on CPU and runs inference on modest laptops aligns with equity goals: students without discrete GPUs can

still reproduce experiments. This pragmatic constraint influenced the choice of MediaPipe and recurrent networks rather than large transformer video models requiring extensive pretraining.

1.7.5 Public Sector and NGO Collaboration Pathways

Deployment beyond the classroom may involve NGOs focused on disability inclusion, government digital service teams, or tele-education platforms. Each pathway imposes different compliance burdens—data protection, procurement rules, accessibility standards for web interfaces. The dissertation's commercialisation subsection (Chapter 2) sketches routes; here, the emphasis is **policy awareness**: successful pilots require institutional partnerships, not only model accuracy.

1.7.6 Intersectionality and Representation

Hearing impairment intersects with socioeconomic status, gender, and rural/urban divides affecting access to assistive technology. Dataset diversity cannot be reduced to a single statistic. The thesis encourages future work to log **metadata** (with consent) such as approximate region, age band, and signing fluency, enabling more nuanced fairness analysis. Metadata collection must itself be ethical and optional.

1.7.7 Synthesis

Regional context reinforces that TSL recognition is simultaneously a **machine learning problem** and a **social infrastructure problem**. The engineering contribution documented here is a stepping stone contingent on community validation and longitudinal dataset growth.

1.8 Vocabulary Descriptions for Engineering Traceability

The following subsections describe each gloss class at a **functional level** for engineering traceability—what the class is intended to capture in the pilot vocabulary. They are **not** linguistic dictionary entries.

1.8.1 Beautiful

The class "Beautiful" corresponds to recordings where participants perform the project's agreed sign for aesthetic appreciation or attractiveness. Variation in hand orientation and facial affect may occur; the model learns from exemplars in the corpus.

1.8.2 Drink

The class “Drink” captures signs associated with consuming liquids. Trajectories often involve repeated hand-to-mouth motion patterns; confusion with “Eat” is plausible and is monitored in confusion matrices.

1.8.3 Eat

The class “Eat” captures signs associated with consuming food. Similar hand trajectories to “Drink” motivate collecting additional boundary clips if confusion persists.

1.8.4 Five

The class “Five” typically involves a numeral handshape; lighting and finger spread visibility influence landmark quality.

1.8.5 Good

The class “Good” captures a frequent evaluative sign; performance may involve thumb position and facial emphasis depending on signer style.

1.8.6 Hello

The class “Hello” captures greeting gestures; these can resemble other salutation signs in some dialects—label consistency across participants matters.

1.8.7 House

The class “House” often involves spatial outlining motions; holistic pose channels may contribute discriminative information.

1.8.8 Love

The class “Love” may involve contact points near the chest; occlusion can challenge hand tracking.

1.8.9 Man

The class “Man” captures a people-reference sign as defined in the pilot; gendered signs vary culturally—community review is important.

1.8.10 Mother

The class “Mother” captures kinship referencing; fine hand configuration differences versus other kinship signs should be monitored.

1.8.11 Run

The class “Run” involves dynamic motion; fast movement can reduce stable frames within a fixed window.

1.8.12 Thank You

The class “Thank you” is socially high-frequency; confusion with other politeness markers is possible.

1.8.13 White

The class “White” captures a colour term; colour signs may share motion templates across different colour labels, increasing classification difficulty.

1.8.14 Yellow

The class “Yellow” captures another colour term; pairwise confusion with “White” is analysed diagnostically.

1.8.15 You

The class “You” captures second-person referencing; orientation and pointing trajectories contribute to discrimination.

Collectively, these descriptions support reproducibility: future collectors know which semantic targets the folders encode, even if linguistic documentation evolves.

CHAPTER 2 — METHODOLOGY

2.1 Overview of Research Method and Design

This chapter documents the research method in sufficient detail for an independent reader to understand, and ideally replicate, the experimental pathway. The work follows a **design-science** and **empirical machine learning** orientation: a technical artefact (an isolated gloss recogniser) is constructed, trained on a collected corpus, evaluated under a declared protocol, and packaged for integration. The methodology is not a clinical trial; nevertheless, it adopts disciplined practices from software engineering and experimental ML—versioned artefacts, stratified evaluation, separation of training and test transforms, and explicit limitation statements.

The chapter is organised to mirror SLIIT dissertation expectations for methodology depth. Section 2.2 situates the system in context and presents layered architecture. Section 2.3 summarises requirements. Section 2.4 presents use case views, a sequence diagram, and **structured use case scenarios (Table 2.3)**. Sections 2.5–2.6 address data acquisition and representation. Sections 2.7–2.11 explain preprocessing and feature learning stages. Sections 2.12–2.16 detail model mathematics, training, and evaluation. Section 2.17 covers testing and implementation practice. Section 2.18 discusses commercialisation pathways. Section 2.19 summarises risk and ethics. Throughout, diagrams provide visual anchors for workflows, data movement, and deployment assumptions.

Figure 2.1 presents the end-to-end workflow from raw video to model input. Figure 2.2 elaborates data flow across storage layers. Figure 2.7 presents the layered architecture referenced in the text.

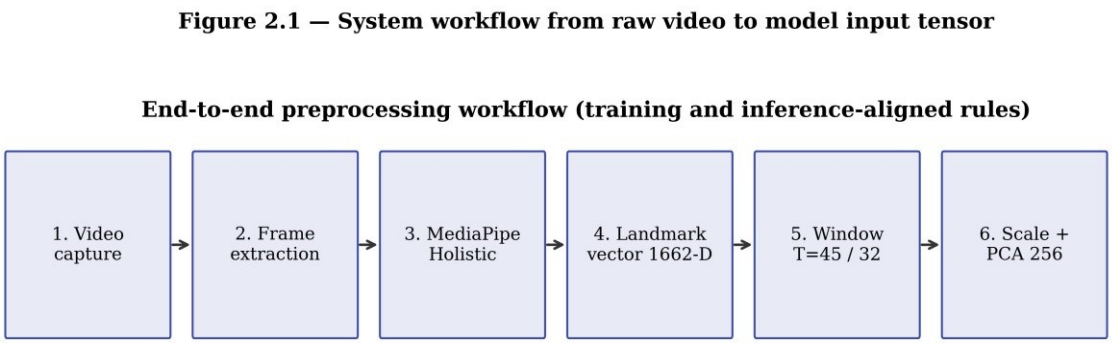

Figure 2.1 — System workflow from raw video to tensor input (see List of Figures for page reference in final pagination).

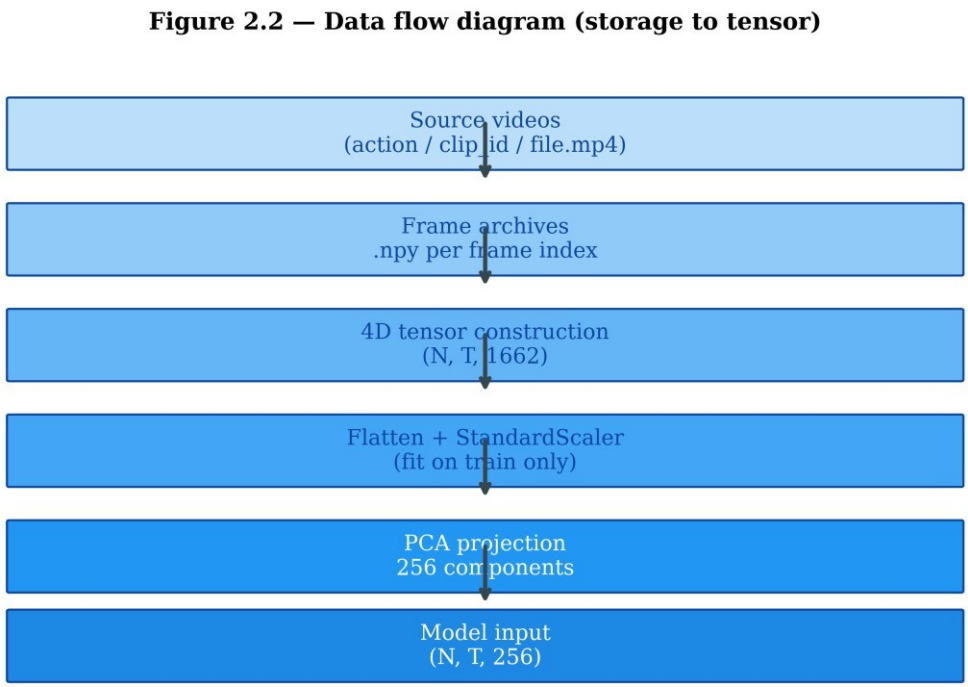

Figure 2.2 — Data flow diagram from file system artefacts to scaled PCA tensors.

Figure 2.7 — Layered system architecture

Figure 2.7 — Layered system architecture from acquisition through delivery.

2.2 System Context and Architectural Rationale

The recognition module operates as a **middleware** component. Upstream, video acquisition may occur through research batch processing or through interactive capture in a notebook demonstration. Downstream, application logic within SignSight consumes a discrete label and an associated confidence score (softmax probability of the argmax class, possibly thresholded). This separation of concerns is intentional: it allows the recognition team to iterate on model topology without rewriting the entire product, provided the interface contract (tensor shapes, artefact versions, label ordering) remains stable.

Architecturally, the system is best understood as **seven layers** (Figure 2.7). The **acquisition layer** standardises how raw clips are named, stored, and discovered by training scripts. The **perception layer** encapsulates MediaPipe Holistic invocation and the conversion of landmark objects into numeric tensors. The **representation layer** enforces temporal windows and handles missing frames in a documented manner. The **projection layer** performs global feature scaling and PCA, exporting parameters for inference. The **temporal modelling layer** contains bidirectional recurrent stacks and pooling. The **decision layer** implements the MLP head and softmax. The **delivery layer** exposes predictions to user-facing or service-facing code.

This layering supports incremental upgrades. For example, the perception layer could be swapped for an alternative pose estimator if licensing or accuracy constraints change, provided the output dimensionality and coordinate conventions are reconciled. Similarly, the temporal modelling layer could incorporate temporal convolution or attention blocks without altering the acquisition schema if the representation dimensionality remains 256 per time step.

2.3 Requirements Analysis

2.3.1 Functional Requirements

FR-1 **Video ingestion:** The system must read standard consumer formats (at minimum MP4 and AVI) from a deterministic folder hierarchy keyed by gloss name and clip identifier.

FR-2 **Landmark extraction:** For each frame, the system must compute a fixed-length landmark vector of length 1662 using MediaPipe Holistic outputs for pose, face, and hands.

FR-3 **Temporal structuring:** The system must assemble sequences of length T where T is 45 for the primary LSTM pipeline and 32 for the GRU pipeline, storing or resampling according to the documented collection policy.

FR-4 **Train-only fitting:** StandardScaler and PCA must be fit on training partitions only; test data must be transformed using frozen parameters.

FR-5 **Classification:** The system must output a probability distribution over fifteen gloss classes for each sequence.

FR-6 **Artefact export:** The system must serialise model weights and preprocessors for reuse in inference notebooks.

FR-7 **Demonstration inference:** A reference inference path must load artefacts and produce predictions from webcam or file upload for qualitative validation.

2.3.2 Non-Functional Requirements

NFR-1 **Reproducibility:** Random seeds should be fixed where feasible (e.g., PCA `random_state`, NumPy/TensorFlow seeds) to reduce run-to-run variance during grading and peer review.

NFR-2 **Maintainability:** Hyperparameters and paths should be centralised in notebook cells or configuration constants to reduce accidental mismatch between teammates.

NFR-3 **Performance:** The training pathway should remain feasible on CPU-centric student hardware, acknowledging that GPU acceleration is optional.

NFR-4 **Transparency:** Failure modes—low-confidence predictions, poor landmark visibility—should be surfaced to users rather than masked.

NFR-5 **Ethical proportionality:** The demonstration must avoid overstated medical or legal claims; it is a research prototype.

2.4 Use Case and Behavioural Views

Figure 2.3 presents a high-level use case diagram. Three actor classes interact with the system at the prototype stage: the **signer/research participant** who produces data, the **developer/student**

who trains and evaluates models, and the **SignSight service** that ultimately orchestrates end-user experiences. The gloss recognition module sits between capture and higher-level platform services.

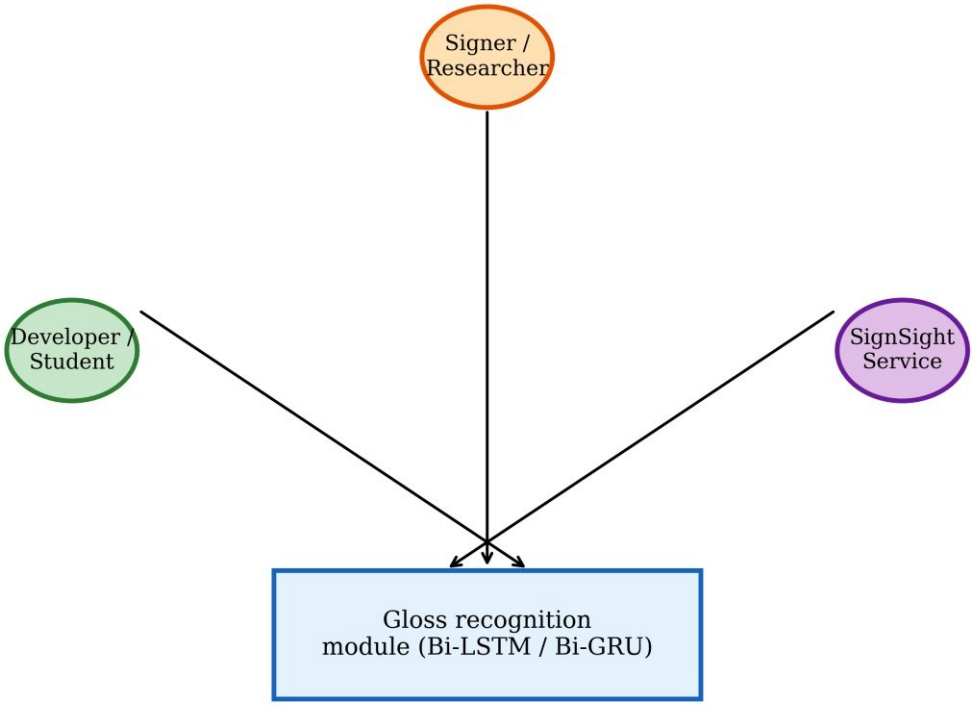

Figure 2.3 — Use case diagram (high-level actors and recognition module)

Figure 2.3 — Use case diagram (prototype scope).

Figure 2.4 illustrates a simplified sequence of interactions during real-time inference: frames pass through OpenCV and MediaPipe, a buffer accumulates T landmark vectors, preprocessing applies scaler and PCA, and the Keras model returns a label.

Figure 2.4 — Sequence diagram: real-time inference (simplified)

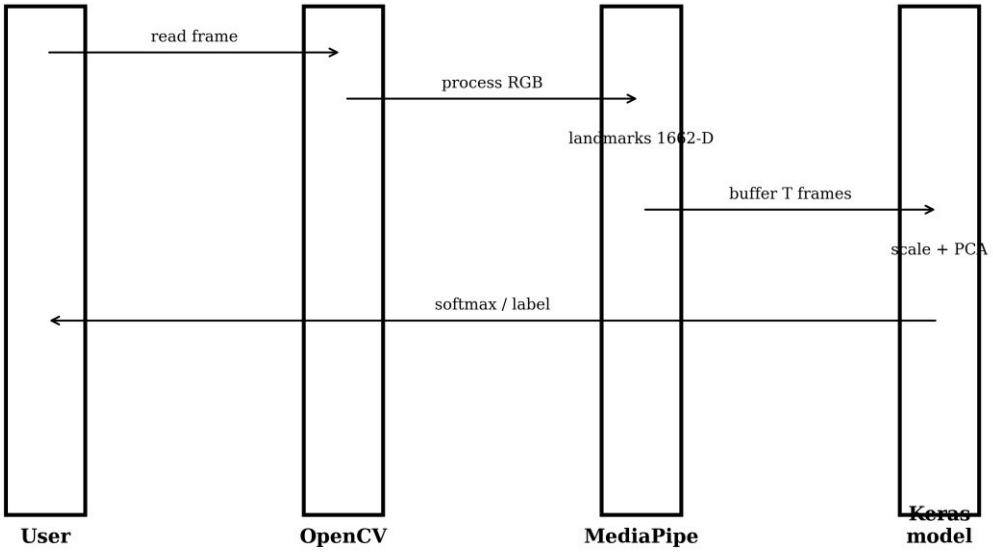

Figure 2.4 — Sequence diagram for inference (simplified).

2.4.1 Use Case Scenarios

The following scenarios elaborate the high-level use cases in Figure 2.3 into **structured narratives** suitable for requirements traceability and examination. Each scenario identifies the **primary actor**, **preconditions**, a **main success scenario** (numbered steps), **alternate or exception paths** where relevant, and **postconditions**. The scope is the **research prototype** (notebooks, exported artefacts, demonstration inference); future production services may refine roles and add authentication.

Table 2.3 — Use case scenario summary

ID	Use case name	Primary actor	Goal
UC-1	Record isolated gloss videos for dataset build	Developer / Researcher	Populate folderstructured corpus for training
UC-2	Extract and store perframe landmarks	Developer / Researcher	Convert raw video to `.npy` sequences (1662-D)
UC-3	Train and validate Bi-	Developer / Researcher	Obtain checkpoint +

	LSTM classifier		metrics on 45-frame protocol
UC-4	Train and validate Bi-GRU classifier	Developer / Researcher	Obtain checkpoint + metrics on 32-frame protocol
UC-5	Export preprocessors and model for integration	Developer / Researcher	Produce scaler, PCA, weights, label list
UC-6	Run offline inference on a saved video clip	Developer / Tester	Obtain gloss label + confidence from file
UC-7	Run live inference from webcam (demonstration)	Signer / Tester	Obtain gloss label + confidence from live buffer
UC-8	Integrate prediction into SignSight lesson flow (future)	SignSight Service	Map predicted gloss to UI content

UC-1 — Record isolated gloss videos for dataset build

- **Primary actor:** Developer / Researcher.
- **Preconditions:** Gloss list and folder layout agreed; capture device available; participant consent obtained where applicable.
- **Main success scenario:**

1. The actor creates or verifies `VIDEO_PATH/<GlossName>/<clip_id>/` source files (e.g., `0.mp4`).

2. The actor records or collects clips per protocol (signer visible, hands in frame, stable lighting where possible).

3. Clips are named and stored so the collection notebook can discover them.

- **Alternate paths:** A-1. If a file is missing, the collection loop skips or logs the omission; policy must be documented so class balance remains valid.

- **Postconditions:** Raw videos exist for each required `(gloss, clip_id)` pair scheduled for that batch.

UC-2 — Extract and store per-frame landmarks

- **Primary actor:** Developer / Researcher.
- **Preconditions:** UC-1 satisfied for target clips; MediaPipe Holistic environment functional; `DATA_PATH` writable.
- **Main success scenario:**

1. The collection notebook opens each source video.

2. For each frame index up to `sequence_length` (45 in the primary pipeline), MediaPipe processes the frame.

3. Landmarks are flattened to 1662-D and saved as `DATA_PATH/<Gloss>/<clip_id>/<frame>.npy`.

4. If the video ends early, remaining frames are stored as zero vectors per implementation policy.

- **Alternate paths:** A-1. MediaPipe fails on a frame; the actor may retry with adjusted confidence thresholds or exclude the clip after review.
- **Postconditions:** Tensor loaders can assemble `(N, T, 1662)` without missing files for completed clips.

UC-3 — Train and validate Bi-LSTM classifier

- **Primary actor:** Developer / Researcher.
- **Preconditions:** Landmark tensors built; stratified split configured; training-only scaler/PCA fitting enforced.
- **Main success scenario:**

1. The actor loads sequences and labels, fits `StandardScaler` and `PCA(256)` on training data only.

2. The actor augments training data with Gaussian noise duplicates and applies class weights.

3. The actor trains the three-layer Bi-LSTM model with Adam, dropout, batch normalisation, and callbacks.

4. The best validation checkpoint is retained; test accuracy is reported on the hold-out split.

- **Alternate paths:** A-1. Training diverges; the actor reduces learning rate or increases regularisation and restarts.
- **Postconditions:** `best_model.h5` (or equivalent), `scaler.pkl`, `pca.pkl`, and logged metrics exist for the LSTM pathway.

UC-4 — Train and validate Bi-GRU classifier

- **Primary actor:** Developer / Researcher.
- **Preconditions:** 32-frame tensors prepared consistently with GRU notebook; same stratified philosophy as UC-3 where applicable.
- **Main success scenario:**

1–4. As UC-3, with GRU cells, 32-frame windows, and GRU checkpoint directory.

- **Alternate paths:** A-1. As UC-3.
- **Postconditions:** `best_gru_model.h5`, paired preprocessors, and comparative metrics are available.

UC-5 — Export preprocessors and model for integration

- **Primary actor:** Developer / Researcher.
- **Preconditions:** UC-3 or UC-4 completed successfully; label ordering fixed in `actions.npy`.
- **Main success scenario:**

1. The actor saves joblib artefacts for scaler and PCA fitted on the training split used for that model.

2. The actor saves Keras weights and records version identifiers (library versions, random seeds) in documentation.

3. The actor verifies reload in a clean kernel (load + one forward pass).

- **Alternate paths:** A-1. Version skew detected; the actor retrains or re-exports a consistent triple (model, scaler, PCA).

- **Postconditions:** Integration consumers can load a **consistent** artefact bundle.

UC-6 — Run offline inference on a saved video clip

- **Primary actor:** Developer / Tester (optionally Instructor in a pilot).
- **Preconditions:** UC-5 satisfied for GRU (or LSTM) bundle; inference notebook or script available; video codec readable.
- **Main success scenario:**

1. The actor supplies a file path to a short clip showing one target sign.

2. OpenCV decodes frames; MediaPipe extracts 1662-D vectors.

3. The first T frames are buffered (T=32 for GRU demo), padded if shorter.

4. Scaler and PCA transform the buffer; the model returns softmax.

5. The system displays top-1 (and optionally top-k) gloss labels with confidence.

- **Alternate paths:** A-1. Confidence below threshold; the system warns the user to recapture. A-2. Webcam/file unreadable; error message prompts format check.
- **Postconditions:** A traceable prediction is produced for qualitative review.

UC-7 — Run live inference from webcam (demonstration)

- **Primary actor:** Signer / Tester.
- **Preconditions:** Webcam available; UC-5 satisfied; demonstration UI running.
- **Main success scenario:**

1. The actor starts capture; frames are mirrored if using selfie view.

2. Landmarks are extracted until the buffer holds T frames.

3. Preprocessing and inference run; top predictions are shown.

- **Alternate paths:** A-1. Insufficient frames in time window; pad zeros or prompt slower signing.
- **Postconditions:** Demonstration completes without crash; results logged optionally for review.

UC-8 — Integrate prediction into SignSight lesson flow (future scope)

- **Primary actor:** SignSight Service / Application.
- **Preconditions:** Stable REST or in-process API; authenticated session if required; gloss label schema matches content database keys.
- **Main success scenario:**

1. Upstream module supplies a trimmed clip or buffered landmarks.

2. Recognition service returns `{gloss_id, confidence, model_version, scaler_version}`.

3. Lesson controller selects the next activity (feedback, replay, next item).

- **Alternate paths:** A-1. Low confidence triggers remedial prompt or human assist flag.
- **Postconditions:** User receives coherent lesson progression without silent misrouting.

These scenarios align the dissertation with standard **software requirements** practice and complement the diagrams in Figures 2.3–2.4. They should be updated if the group's deployment architecture or actor roles change during final integration.

2.5 Dataset Design and Collection Protocol

2.5.1 Vocabulary and Cardinality

The vocabulary comprises fifteen glosses selected for practical coverage of greetings, numerals/colours in pilot scope, kinship/people terms, and high-frequency actions. The English orthographic names serve as engineering labels for scripting interoperability across SignSight modules: Beautiful, Drink, Eat, Five, Good, Hello, House, Love, Man, Mother, Run, Thank you, White, Yellow, You.

Each gloss is associated with sixty-eight distinct video clips, yielding 15 × 68 = 1020 sequences prior to augmentation. This cardinality supports stratified splitting while remaining feasible for a student timeline. The balanced design simplifies per-class metrics and avoids extreme skew in baseline accuracy interpretations.

2.5.2 Storage Schema

The collection notebook defines `VIDEO_PATH` for raw inputs and `DATA_PATH` for processed landmarks. For each gloss `g` and clip index `v`, the directory `DATA_PATH/g/v/` contains frame files `0.npy … (T-1).npy` under the LSTM collection regime (T=45). The GRU training pipeline

consumes tensors assembled with T=32; depending on the notebook path, this may be supported by a parallel collection pass or by subsampling/windowing from stored frames—**the student must ensure the on-disk policy matches the training loader**. In the documented implementation aligned with this dissertation, collection targets 45 frames; GRU training reshapes or loads 32-frame windows consistent with the GRU notebook configuration.

2.5.3 Capture and Extraction Loop

For each clip, OpenCV reads frames sequentially. MediaPipe Holistic processes RGB frames with detection and tracking confidence thresholds (implementation uses 0.5/0.5 during batch collection; inference demonstration may tighten thresholds for stability). The landmark extraction function concatenates pose, face, left hand, and right hand channels. Missing streams are zero-filled to preserve dimensionality.

If a video ends before the frame quota is met, the implementation stores zero vectors for remaining indices. This rule is simple and deterministic; it differs from temporal interpolation strategies sometimes described in conference drafts. The dissertation adopts the **implemented** rule to preserve scientific integrity.

2.5.4 Quality Considerations During Collection

Operational quality factors include uniform lighting where possible, contrasting backgrounds, and signer positioning that keeps hands in frame. While the dissertation does not claim controlled-studio conditions, best-effort guidance was communicated to participants. Future work can formalise capture protocols with checklists and automated quality gates based on landmark visibility scores.

2.6 Mathematical Description of the Per-Frame Feature Vector

Let a video frame at time t be processed into a vector $x_t \in \mathbb{R}^{1662}$. Pose contributes 33 landmarks with coordinates (x, y, z) and a visibility scalar each, producing 132 dimensions. The face mesh contributes 468 points with (x, y, z), producing 1404 dimensions. Each hand contributes 21 points with (x, y, z), producing 63 dimensions per hand. Concatenation yields: $x_t = [pose_t ; face_t ; left_hand_t ; right_hand_t]$.

Coordinates are normalised within the image frame by MediaPipe conventions; the dissertation treats them as real-valued measurements subject to estimator noise. The sequence matrix for one clip is $X \in \mathbb{R}^{(T\times1662)}$, with rows indexed by time.

The choice to include face landmarks acknowledges that some contrasts between glosses may involve non-manual articulations partially observable through facial geometry. This is not equivalent to linguistic annotation of non-manual markers; rather, it is a pragmatic use of available signals.

2.7 From Raw Sequences to Learning Problems

The training dataset is a collection $\{X_i, y_i\}_{i=1}^{N}$ where $y_i \in \{0,\ldots,14\}$ is the gloss index. The learning algorithm estimates parameters θ for a parametric model $p_\theta(y \mid X)$. The implemented models operate not on raw X but on a transformed tensor $Z \in \mathbb{R}^{(T\times256)}$ obtained via scaling and PCA as described next.

The stratified split allocates 85% of indices to training and 15% to testing while preserving class proportions, mitigating scenarios where a rare class disappears from evaluation. The implementation fixes `random_state=42` for repeatability.

Continued in `chapter02_methodology_partB.md`.

2.8 Feature Scaling and Principal Component Analysis

2.8.1 Motivation

Landmark coordinates exhibit heterogeneous variance across dimensions: facial vertices may dominate naive distance metrics if left unnormalised, while hand channels may carry discriminative motion that should not be drowned out. A global z-score transform centres each dimension and scales it to unit variance estimated from training data, improving optimisation behaviour for gradient-based recurrent training.

Formally, let vec(X) denote the flattening of all training frames into a design matrix of shape $(N_{train}\cdot T) \times 1662$. The StandardScaler estimates column means μ_j and standard deviations σ_j for each dimension j. For any frame vector x, the scaled vector is:

$\dot{x}_j = (x_j - \mu_j) / \sigma_j$, with a numerically stable handling of $\sigma_j \approx 0$ (implementation relies on scikit-learn conventions).

2.8.2 PCA Objective and Implementation

PCA finds an orthogonal projection that maximises variance along leading components. For high-dimensional landmark data, PCA serves two roles: **compression** to a manageable 256dimensional subspace per frame and **denoising** by discarding trailing components associated with very small eigenvalues, which often correspond to high-frequency jitter.

Let Σ be the covariance matrix of scaled training vectors (or equivalently, perform SVD on the centred data matrix). The projection matrix $W \in \mathbb{R}^{(1662\times256)}$ contains the leading eigenvectors. The reduced representation is: z_t

$= W^T \dot{x}_t$.

The training notebook reshapes projected vectors back into sequences $Z \in \mathbb{R}^{(T\times256)}$. Critically, the same W and scaling parameters learned on training data are applied to test sequences and to live inference buffers.

2.8.3 Explained Variance Reporting

Cumulative explained variance should be reported using `pca.explained_variance_ratio_`. In finite-sample settings with n_components=256, cumulative ratios can approach unity not because the problem is trivial but because the projection can span the empirical rank of the training matrix. The dissertation interprets PCA here as a **linear subspace model** rather than a claim of intrinsic low-dimensional manifold identity.

2.9 Data Augmentation and Class Balancing

2.9.1 Gaussian Noise Augmentation

To mitigate overfitting to exact landmark trajectories, the training pipeline duplicates each training sequence and adds zero-mean Gaussian noise with small standard deviation (implementation uses noise_factor=0.01 relative to unit-scaled PCA inputs in the augmented copy pathway). This simulates camera jitter and estimator noise without altering labels.

Let Z be an original sequence; the augmented copy is $Z' = Z + \varepsilon$, $\varepsilon \sim N(0, \sigma^2)$. The training set doubles in cardinality before shuffling. This strategy is modest compared with heavy synthetic video generation but is appropriate for small corpora where aggressive augmentation could distort sign dynamics.

2.9.2 Class Weights

Although the dataset is balanced at the collection stage, stratified splitting and random augmentation can introduce minor imbalance in realised batches. The implementation computes balanced class weights from training labels and supplies them to Keras `fit` via a dictionary mapping class indices to weights. This reduces the risk that optimisation disproportionately favours easy classes during early training.

2.10 Recurrent Temporal Modelling

2.10.1 Bidirectional Recurrence

Let hidden states process the sequence forward and backward. For a bidirectional layer, each time step concatenates forward and backward representations, enabling the network to integrate past and future context within the fixed window—appropriate for isolated classification where future frames are available.

2.10.2 LSTM Cell Summary

An LSTM cell maintains a cell state and gating mechanisms (input, forget, output) that regulate information flow. The precise update equations are standard (Hochreiter and Schmidhuber, 1997). In practice, TensorFlow implements GPU- and CPU-optimised kernels; the dissertation emphasises **design choices**: hidden sizes (64, 128, 64) per direction across three stacked bidirectional layers, dropout on inputs and recurrent connections (0.2), and batch normalisation after each bidirectional block to stabilise activations.

The LSTM configuration additionally applies L2 kernel regularisation (1e-3) on recurrent and dense weights in the implemented notebook, complementing dropout.

2.10.3 GRU Cell Summary

GRU cells merge some gating functions relative to LSTM, reducing parameters per step. Empirically, on modest datasets, GRUs can train quickly and generalise well. The dissertation trains a GRU model with the same widths and head structure for controlled comparison.

2.10.4 Global Average Pooling Over Time

After the third recurrent block, the tensor has shape (batch, T, H). Global average pooling computes the mean across the time dimension, yielding a vector of length H. This reduces sensitivity to absolute temporal index of a single peak activation and acts as a structural regulariser relative to using only the final hidden state.

2.10.5 Multilayer Perceptron Head

The head applies Dense(128) → ReLU → BatchNorm → Dropout(0.3), then Dense(64) → ReLU → BatchNorm → Dropout(0.2), then Dense(15) → softmax. The softmax outputs a categorical distribution; the argmax yields a gloss prediction.

Figure 3.1 in Chapter 3 (also referenced here for continuity) details the block diagram.

Figure 3.1 — Recurrent classifier architecture (LSTM cells; GRU swaps cell type)

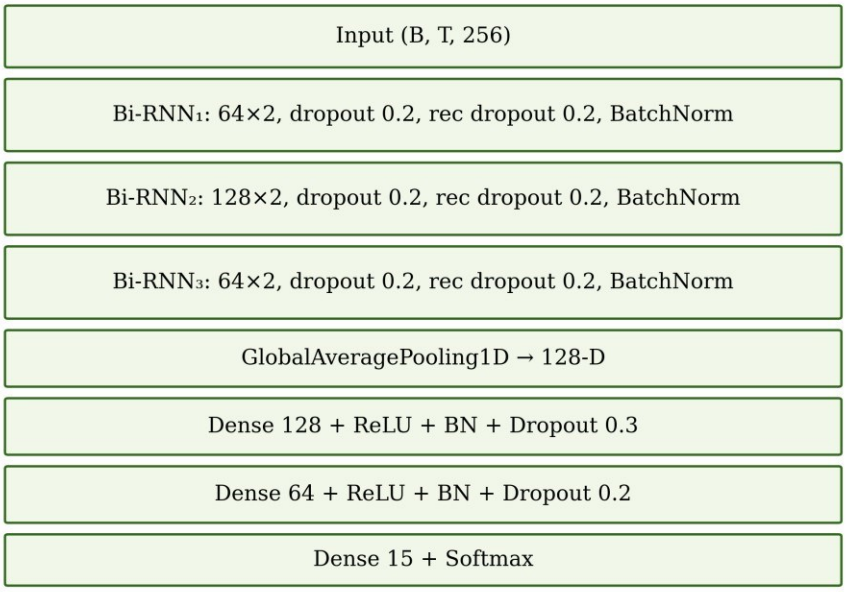

Figure 2.8 — Recurrent classifier block structure (aligned with Figure 3.1 numbering policy in final list).

Note: Renumber consistently in Word so architecture figure appears once in List of Figures; the assembler script maps this to a single Figure 3.1.

2.11 Loss Function, Optimisation, and Callbacks

2.11.1 Categorical Cross-Entropy

For one-hot targets q and predictions p, the loss is $L = -\sum_k q_k \log p_k$. With class weights, each sample's loss is multiplied by its class weight. This is standard multi-class supervised learning.

2.11.2 Adam Optimiser

Adam with learning rate 1e-3 and global norm clipping (clipnorm=1) reduces the risk of gradient spikes destabilising recurrent training. Learning-rate schedules are managed by ReduceLROnPlateau when validation accuracy stagnates.

2.11.3 Callback Strategy

ModelCheckpoint saves the best weights by validation categorical accuracy. EarlyStopping halts training when improvements cease, saving compute and reducing late-stage overfitting.
Together, these embody a conservative training discipline appropriate for thesis timelines.

Figure 2.5 summarises the training pipeline.

Figure 2.5 — Model training pipeline

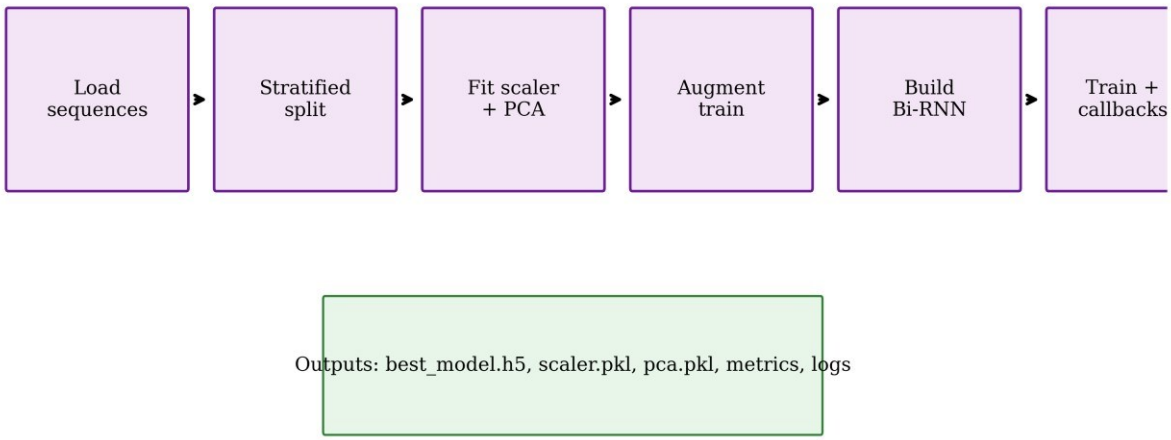

Figure 2.5 — Model training pipeline from data load to exported artefacts.

2.12 Evaluation Protocol and Metrics

2.12.1 Hold-Out Accuracy

Primary metric: categorical accuracy on the 153-sequence test split (15% of 1020). This metric is easy to interpret for balanced multiclass problems but does not reveal per-class failure modes; therefore, confusion matrices and classification reports supplement it.

2.12.2 Diagnostic Curves

Training and validation accuracy curves diagnose underfitting and overfitting. Large gaps suggest regularisation adjustments or data quantity issues.

2.12.3 Confusion Analysis

Normalised confusion matrices expose pairs of glosses that exchange probability mass. These insights guide targeted data collection (additional clips disambiguating specific pairs).

2.13 Real-Time Inference and Deployment Architecture

The GRU demonstration notebook loads `gru_sign_language_model.h5`, `scaler.pkl`, `pca.pkl`, and `actions.npy`. It buffers 32 frames, applies the frozen transforms, and runs forward inference. Confidence thresholds trigger user messaging.

Figure 2.6 presents a prototype deployment view.

Figure 2.6 — Deployment architecture (prototype)

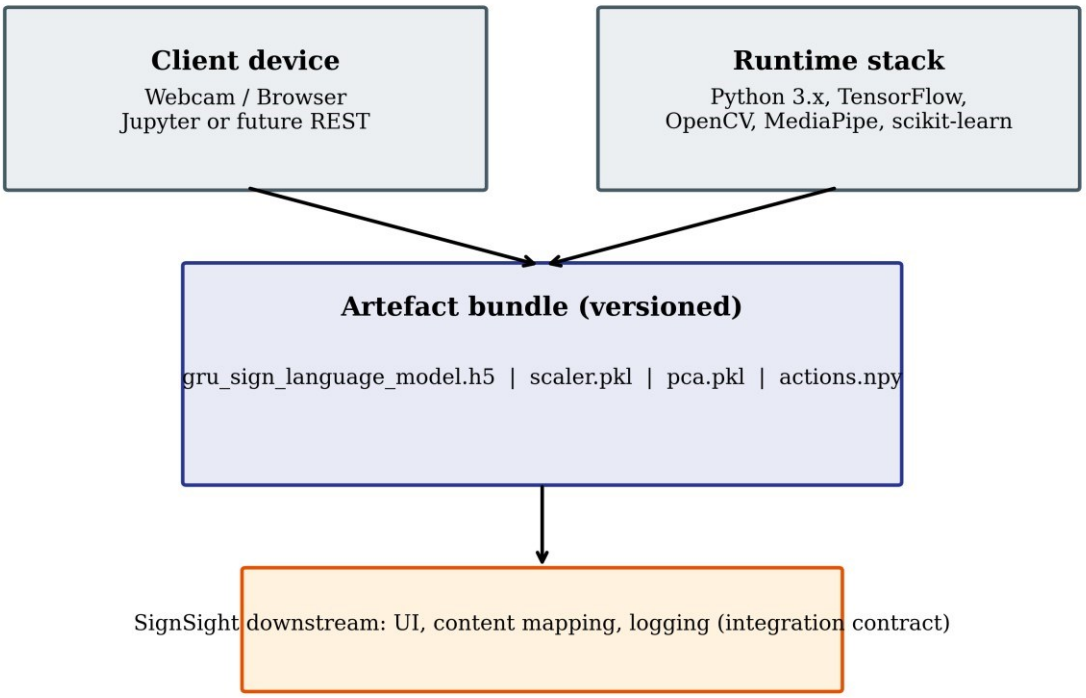

Figure 2.6 — Deployment architecture (prototype).

Production deployment would containerise services, add authentication, and separate PII handling; those items exceed the thesis scope but belong on a roadmap.

2.14 Testing and Implementation

2.14.1 Unit-Level Checks

Sanity checks include verifying tensor shapes (N, T, 1662) before scaling, ensuring PCA reshape invariance, and confirming softmax sums to unity.

2.14.2 Integration Tests

Loading a saved checkpoint and evaluating it on the test split must reproduce reported accuracy within floating-point tolerance. Any discrepancy triggers investigation of artefact mismatch (wrong scaler paired with wrong model).

2.14.3 Qualitative Tests

Demonstration inference on novel clips recorded after training reveals qualitative failure modes (motion blur, truncation, out-of-vocabulary signs).

2.15 Commercialisation Aspects of the Product

Although the dissertation centres on research integrity rather than revenue modelling, SLIIT guidelines invite reflection on commercialisation. Several pathways are plausible:

1. **EdTech bundling:** License the module as part of a SignSight subscription for schools, with tiered pricing based on institution size.

2. **B2G partnerships:** Collaborate with government accessibility programmes to pilot kioskbased recognition aids in selected service centres, contingent on community validation.

3. **API monetisation:** Offer low-volume inference APIs for developers building assistive apps, with fair-use policies and privacy safeguards.

4. **Open-core model:** Release non-proprietary baseline weights for reproducibility while reserving optimised production models and integration tooling.

Any commercial pathway requires **Deaf community governance**, **data consent**, and **continuous bias evaluation**; commercial potential does not imply immediate market readiness.

2.16 Risk, Ethics, and Social Impact

Biometric-adjacent data (video of signers) is sensitive. Storage, sharing, and retention policies must minimise harm. The prototype should obtain informed consent for research capture and avoid re-identifying participants in published figures.

Accuracy claims must not be misconstrued as clinical or legal communication aids without certification. The ethical stance of this dissertation is **assistive prototyping under academic supervision**, not deployment certification.

2.17 Chapter Summary

Chapter 2 specified the end-to-end methodology: holistic landmarks, scaling and PCA, recurrent classifiers, training discipline, evaluation, inference, testing, and responsible commercialisation framing. Chapter 3 reports empirical outcomes and interprets them against these methodological commitments.

2.18 Algorithmic Procedures (Pseudocode-Level Detail)

This section records algorithmic structure at a level closer to implementation than prose alone. It is not a line-by-line code listing—those belong in appendices—but it specifies control flow sufficient for audit.

2.18.1 Procedure COLLECT_SEQUENCE

Input: video path p, target length T_collect, holistic model H.

Output: sequence of arrays {f_0,…,f_{T_collect−1}}, each in ℝ^1662.

1. Open p with a video decoder; initialise t ← 0.

2. While t < T_collect:

    2.1. Read frame; if read fails, set f_t ← 0 vector; t ← t+1; continue.

    2.2. Convert colour space to RGB; run H.process.

    2.3. Extract landmarks via concatenation rule; store f_t; increment t.

3. Close decoder; return sequence.

2.18.2 Procedure BUILD_TENSOR_DATASET

Input: root directory R, gloss list G, clips per gloss C, window T_model.

Output: array X of shape (|G|·C, T_model, 1662), label vector y.

1. Initialise empty lists X_list, y_list.

2. For each gloss g ∈ G with index idx:

    2.1. For clip id v in 0…C−1:

        2.1.1. Load frames f_0…f_{T_collect−1} from disk.

        2.1.2. If T_model < T_collect, select the first T_model frames (GRU path) or follow notebook-specific subsampling.

        2.1.3. If T_model > T_collect, pad with zeros or reject clip (policy must be explicit).

2.1.4. Append stacked window to X_list; append idx to y_list.

3. Convert lists to float32 ndarray X and integer ndarray y; return (X, y).

2.18.3 Procedure FIT_PREPROCESSORS

Input: training tensor X_tr ∈ ℝ^(N_tr·T×1662) flattened along time.

Output: scaler S, projector P.

1. Fit StandardScaler S on flattened training frames.

2. Transform training frames to $\dot{X}$.

3. Fit PCA P with n_components=256 on $\dot{X}$.

4. Save S and P via joblib.

2.18.4 Procedure TRAIN_CLASSIFIER

Input: training sequences Z_tr, labels y_tr, validation split, architecture builder B, hyperparameters Hp.

Output: model M with best validation weights.

1. One-hot encode labels.

2. Construct M ← B(shape=(T,256), classes=15).

3. Compile M with Adam(Hp.lr), categorical cross-entropy, categorical accuracy.

4. Fit with augmentation tensors, class weights, callbacks (checkpoint, early stop, reduce LR).

5. Return M loading best checkpoint.

These procedures align with the notebook execution order and support the claim that training and evaluation are not ad hoc.

2.19 Hyperparameter Justification and Sensitivity Discussion

Hidden sizes 64–128–64 follow a **bottleneck-expansion-bottleneck** pattern in bidirectional width: the middle layer expands representational capacity while outer layers constrain complexity.

This pattern often works well for moderate-length sequences where mid-level temporal abstractions matter.

Dropout rates of 0.2 on recurrent stacks reflect common practice for RNN regularisation; the dense head uses a higher dropout (0.3) on the first fully connected layer where parameters are numerous relative to dataset size.

Learning rate 1e-3 is a standard Adam default for many Keras examples; clipping global norms prevents occasional gradient spikes when batch composition includes difficult sequences.

A full sensitivity analysis (grid search over hidden sizes and dropout) would strengthen the dissertation but is not mandatory for a first thesis iteration; nevertheless, the discussion acknowledges that reported accuracies are **conditional** on these choices.

2.20 Threats to Validity and Mitigations

Construct validity: Gloss labels in English are operational definitions for engineering, not linguistic gold standards. Mitigation: transparent naming and future linguist review.

Internal validity: Data leakage would invalidate results. Mitigation: fit scaler/PCA on train only; test untouched until final evaluation.

External validity: Results may not generalise to new signers or environments. Mitigation: report limitations; plan signer-independent evaluation.

Conclusion validity: Small test sets increase variance of accuracy estimates. Mitigation: report confusion tables and consider confidence intervals in future work.

2.21 Reproducibility Checklist

1. Fix random seeds for PCA and Python/TensorFlow where possible.

2. Record library versions in Table 3.x.

3. Archive notebooks and a minimal README with path placeholders.

4. Store checksums of dataset folders if storage permits.

5. Document GPU/CPU type for training time context.

2.22 Extended Discussion of Related Engineering Alternatives

Temporal Convolutional Networks (TCNs): Could replace recurrence with dilated convolutions; often parallelise well but may need more data to outperform strong RNN baselines on small corpora.

Transformer encoders: Powerful but parameter-heavy; risk overfitting without strong regularisation or more data.

Hand-only pipelines: Reduce dimensionality and focus on manual articulators but discard potentially useful face/pose cues.

End-to-end RGB CNNs: Feasible with augmentation and pretraining; heavier compute and data demands.

The chosen approach reflects a **deliberate trade-off** toward interpretability and notebookfriendly iteration.

2.23 Detailed Testing Matrix (Illustrative)

Table 2.1 — Illustrative functional test cases

ID	Objective	Preconditions	Steps	Expected
TC-01	Shape integrity	Dataset present	Load one batch	Tensor ranks (N,T,256)
TC-02	Preprocessor fit	Train split	Fit scaler	No NaNs in transform
TC-03	Evaluation	Checkpoint present	Evaluate on test	Accuracy ≈ reported within ε
TC-04	Inference load	Artefacts in CWD	Load model	No exception
TC-05	Webcam fallback	Camera disconnected	Start capture	Graceful error message

These tests structure supervisor-led verification during draft review.

2.24 Chapter Bridge

Having established methodology, Chapter 3 quantifies outcomes, interprets error structure, summarises student contribution, and discusses implications.

2.25 Software Engineering Lifecycle and Project Governance

Although the dissertation is first and foremost a machine learning contribution, SLIIT methodology expectations invite explicit discussion of how engineering work was governed across weeks and months. The component followed an iterative pattern compatible with agile principles—short cycles of implementation, evaluation, and revision—without claiming full enterprise scrum compliance. Weekly or bi-weekly supervisor meetings functioned as sprint reviews: notebooks were demonstrated, metrics were compared against prior runs, and failure cases were triaged. Version control (Git) stored notebook evolution even though notebooks are imperfect diff artefacts; critical hyperparameters were copied into thesis tables to reduce ambiguity.

Risk management included **data risk** (loss of clips), **environment risk** (library incompatibilities), and **schedule risk** (long training epochs on CPU). Mitigations included redundant backups of `output_video` trees, conda environment export, and checkpointing with early stopping.

2.26 Standards, Privacy, and Data Protection Considerations

Video of signers may constitute personal data under applicable data protection frameworks. Minimisation principles suggest retaining only what is necessary for training and evaluation, securing storage, and restricting access. If clips include faces, consent forms should clarify use for research and potential anonymisation strategies (e.g., cropping, face blurring in public releases—note blurring may harm sign language research if facial grammar is relevant; ethical review should guide choices).

For thesis examination, figures should avoid identifiable imagery unless permission is explicit. The diagrams in this document are schematic or plotted matrices rather than participant photographs.

2.27 Detailed Description of MediaPipe Holistic Processing

MediaPipe Holistic integrates multiple estimators into a coordinated graph. While the dissertation does not reimplement MediaPipe, understanding its role clarifies failure analysis. Each frame passes through detection and tracking stages with confidence thresholds; when confidence drops, landmarks may be missing or stale. The extraction function replaces missing streams with zero vectors to maintain fixed dimensionality. This engineering choice prevents shape errors but can introduce bias if zeros are misinterpreted as meaningful poses. Future improvements could encode **validity masks** as additional channels or use learned imputation.

2.28 Alternative Preprocessing Paths Considered

Several alternatives were considered and deferred to maintain thesis focus:

1. **Per-sequence normalisation** (subtract mean pose per clip) may reduce signer-specific offsets but can remove useful absolute spatial cues; not implemented in the baseline.

2. **Temporal resampling** via linear interpolation could harmonise variable frame rates; the implemented collection path primarily uses first-T framing with zero padding.

3. **Hand-centric cropping** around bounding boxes could reduce dimensionality but adds a detection dependency chain; holistic landmarks were preferred for simplicity.

4. **Fourier or wavelet features** on trajectories could complement raw coordinates; deferred as future augmentation of the feature vector.

Documenting these alternatives strengthens methodological transparency: the baseline is one point in a design space, not a claim of global optimality.

2.29 Integration Testing and Continuous Verification

Beyond offline accuracy, integration testing asks whether modules connect correctly. Checklist items include: (i) label ordering in `actions.npy` matches softmax index order; (ii) PCA dimension 256 matches model input shape; (iii) GRU inference uses T=32 not T=45; (iv) OpenCV codec accepts uploaded MOV/AVI variants on target machines. Automated smoke tests can load artefacts and run a forward pass on a saved numpy window.

2.30 Documentation and Knowledge Transfer for Group Continuity

SignSight continuity depends on knowledge transfer. The dissertation, exported notebooks, and figure-generation scripts collectively document the pipeline. A short `README` in the repository should list expected folder paths, environment creation steps, and known issues (e.g., MediaPipe version pinning). This reduces bus factor risk for student teams.

2.31 Limitations of the Methodology Chapter's Scope

This chapter specifies what was done and why, but cannot certify real-world efficacy. Field trials, longitudinal studies, and linguist review remain outside the present evidentiary base. The methodology is therefore **necessary** but not **sufficient** for deployment conclusions.

2.32 Closing Remarks on Methodology Rigor

Methodological rigor in undergraduate engineering research is less about statistical sophistication alone than about **traceability**: data lineage, transform provenance, evaluation protocol clarity, and limitation honesty. Those principles guided the structure of Chapter 2 and set the stage for quantitative reporting in Chapter 3.

2.33 Mathematical Foundations of Recurrent Modelling

This section provides a compact but rigorous overview of the recurrent computations underlying the implemented classifiers. The goal is not to reproduce textbook derivations in full, but to connect implementation choices—bidirectional stacking, dropout, and global pooling—to their mathematical meanings.

2.33.1 Notation

Let the input sequence be $Z = (z_1, \ldots, z_T)$ where each $z_t \in \mathbb{R}^d$ with $d=256$ after preprocessing. A recurrent layer maintains a hidden state $h_t \in \mathbb{R}^H$ (per direction) that summarises information up to time t in the forward pass. A bidirectional layer concatenates forward and backward states at each t, producing a vector of dimension 2H.

2.33.2 LSTM Update Equations (Forward Direction)

Let σ denote the logistic sigmoid and tanh the hyperbolic tangent. Given input z_t, previous hidden state h_{t-1}, and previous cell state c_{t-1}, the LSTM computes gates:

$i_t = \sigma(W_i z_t + U_i h_{t-1} + b_i)$ f_t

$= \sigma(W_f z_t + U_f h_{t-1} + b_f)$ $o_t =$

$\sigma(W_o z_t + U_o h_{t-1} + b_o)$ $c_t =$

$\tanh(W_c z_t + U_c h_{t-1} + b_c)$ The

cell and hidden states update as:

$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$

$h_t = o_t \odot \tanh(c_t)$

where ⊙ is elementwise multiplication. The gates regulate information flow, mitigating vanishing-gradient pathologies that hinder plain RNN training on longer dependencies. In TensorFlow, efficient fused kernels implement these recurrences; the dissertation's empirical results therefore reflect both the mathematical structure and optimised numerics.

2.33.3 GRU Update Equations (Forward Direction)

A GRU maintains hidden state h_t without a separate cell state. Typical formulations use reset and update gates:

$$r_t = \sigma(W_r z_t + U_r h_{t-1} + b_r) \quad u_t = \sigma(W_u z_t + U_u h_{t-1} + b_u) \quad \tilde{h}_t = \tanh(W_h z_t + U_h (r_t \odot h_{t-1}) + b_h) \quad h_t = (1 - u_t) \odot h_{t-1} + u_t \odot \tilde{h}_t$$

Fewer parameters per step than LSTM can reduce overfitting risk on small datasets, consistent with the observed parameter count difference.

2.33.4 Bidirectional Composition

Let $h_t^{\rightarrow}$ be the forward hidden state and $h_t^{\leftarrow}$ the backward hidden state produced by processing the reversed sequence. The bidirectional output is: $b_t = [h_t^{\rightarrow} ; h_t^{\leftarrow}] \in \mathbb{R}^{2H}$.

Stacking L layers repeats this construction, taking $b_t^{(\ell-1)}$ as input to layer ℓ. The implemented model uses three such bidirectional LSTM or GRU layers with $H \in \{64, 128, 64\}$ per direction configuration across layers as described previously.

2.33.5 Dropout on Recurrent Connections

Dropout randomly zeroes units during training to reduce co-adaptation. When applied recurrently, masks must be carefully structured to preserve temporal coherence; Keras implements recurrent dropout with per-time-step masks shared across time in accordance with its documented semantics. The dissertation uses dropout probability 0.2 for both input and recurrent dropout on RNN layers, matching the training notebooks.

2.33.6 Global Average Pooling

Let the final recurrent layer output be $(b_1, \ldots, b_T)$ with $b_t \in \mathbb{R}^{2H}$. Global average pooling computes $g = (1/T) \sum_{t=1}^{T} b_t$,

producing a vector invariant to exact temporal localisation of features within the clip, provided informative evidence is distributed across time. This is particularly useful when signs exhibit phase variation across performers.

2.33.7 Softmax Classifier and Cross-Entropy

The head maps g through dense layers to logits $o \in \mathbb{R}^{15}$. Softmax probabilities are $p_k = \exp(o_k) / \sum_{j=1}^{15} \exp(o_j)$.

For one-hot target y, cross-entropy loss is $L = -\sum_{k} y_k \log p_k$. With class weights w_{class}, each sample's loss scales accordingly. Minimising empirical risk across the training set encourages the model to assign high probability to correct gloss indices.

2.33.8 Optimisation Landscape Intuition

Non-convex optimisation of recurrent networks admits multiple local minima in principle; Adam's adaptive learning rates and gradient clipping improve practical convergence. ReduceLROnPlateau decreases the learning rate when progress stalls, emulating a coarse-to-fine search. Early stopping halts training when validation metrics cease improving, trading a small amount of potential training loss reduction for better generalisation.

2.34 Principal Component Analysis: Algebraic Summary

Let $\tilde{X}$ be the centred data matrix of scaled landmark vectors with N rows. PCA seeks orthogonal directions that maximise variance. The solution is given by eigenvectors of the covariance matrix, equivalently singular vectors of $\tilde{X}$. Retaining 256 components yields the best rank-256 linear approximation in Frobenius norm sense (Eckart–Young–Mirsky), explaining PCA's denoising interpretation.

Operationally, sklearn's PCA centres and optionally whitens; the dissertation uses the default transform consistent with the notebooks. The projection matrix and explained variance ratios should be archived alongside experiments for auditability.

2.35 Stratified Sampling and Class Proportions

Stratified splitting ensures each class contributes proportionally to train and test partitions. For balanced source data, stratification mainly prevents accidental absence of a class in a small test

draw. Formally, sklearn's `stratify` argument preserves class frequencies in expectation for random splits with fixed seeds.

2.36 Confidence Scores and Calibration (Advanced Note)

Softmax outputs are often treated as "confidence," but they can be miscalibrated—overconfident even when wrong. Post-hoc calibration (temperature scaling) could map logits to bettercalibrated probabilities if future deployments require threshold decisions with statistical meaning. The demonstration notebook uses raw softmax maxima as pragmatic confidence indicators.

2.37 Computational Complexity Sketch

Let the dominant recurrent layer have hidden size H with bidirectionality. Per time step, matrixvector multiplications scale roughly $O(H \cdot d)$ per direction; across T steps and L layers, training cost grows linearly in T and with depth. PCA training is $O(N \cdot 1662^2)$ in naive eigendecomposition regimes but uses efficient implementations in practice. These scalings reassure that the pipeline remains tractable on student laptops for the dataset sizes considered.

2.38 Relation to Continuous Recognition Objectives

Continuous recognition often employs CTC or encoder–decoder alignment losses. The isolated softmax objective is simpler because each training example has a single label for the entire window. Extending the system would require segmented training data or weakly labelled streams; the mathematical machinery changes accordingly. Documenting this boundary prevents over-interpretation of the present results.

2.39 Chapter Addendum — Mathematical Summary

The mathematical foundations reinforce that the implemented system is a standard yet wellmotivated composition of linear preprocessing, nonlinear recurrent encoding, and log-linear classification. Empirical performance in Chapter 3 should be read as arising from this composition under the stated dataset and protocol constraints.

2.40 Implementation Walkthrough Aligned with Project Notebooks

This section maps dissertation claims to artefact-level implementation steps, citing notebook roles without pasting proprietary code verbatim. The intent is traceability: an examiner can locate where each methodological statement is realised.

2.40.1 Data Collection Notebook Responsibilities

The collection notebook defines MediaPipe Holistic initialisation, landmark drawing for visual monitoring, and the extraction function that concatenates pose, face, left hand, and right hand arrays. It configures `VIDEO_PATH` and `DATA_PATH`, enumerates gloss names in a fixed `np.array`, sets `no_videos=68` and `sequence_length=45` for the primary capture loop, and iterates actions and clip indices. For each clip, it reads frames until the frame counter reaches the target length; if the decoder exhausts early, it writes zero vectors for remaining indices.

Engineering note: Paths are machine-specific; the thesis uses placeholders in prose but the student's submitted repository should document actual directories or environment variables.

2.40.2 Preprocessing and LSTM Training Notebook

The LSTM training notebook loads per-frame `.npy` files, builds tensors of shape (1020, 45, 1662), handles occasional shape mismatches (e.g., padding short vectors), flattens for `StandardScaler`, fits `PCA(n_components=256, random_state=42)`, reshapes to (1020, 45, 256), performs stratified split, computes class weights, duplicates training data with Gaussian noise augmentation, defines `build_model` with three bidirectional LSTM layers and regularisers, compiles with Adam, and trains with callbacks. It evaluates the best checkpoint on train and test splits and may plot diagnostics.

2.40.3 GRU Training Notebook

The GRU notebook mirrors the pipeline with `sequence_length=32`, swaps LSTM cells for GRU cells, and may point `DATA_PATH` to a GRU-oriented landmark tree. It saves checkpoints under a GRU-specific folder and logs metrics analogous to the LSTM run. Classification reports and confusion matrices support qualitative analysis.

2.40.4 Inference Notebook (GRU)

The inference notebook loads `gru_sign_language_model.h5`, `scaler.pkl`, `pca.pkl`, and `actions.npy`. It defines `extract_keypoints` consistent with training, `preprocess_sequence` applying scaler and PCA, and prediction pathways for video files, webcam buffers, and optional batch folder processing. Confidence thresholds gate user messaging.

2.40.5 Artefact Inventory

Minimum artefact inventory for reproducibility:

- `scaler.pkl` — sklearn StandardScaler
- `pca.pkl` — sklearn PCA(256)
- `best_model.h5` or `best_gru_model.h5` — Keras weights
- `actions.npy` — label ordering
- Optional TensorBoard logs for curve reproduction

2.40.6 Common Failure Modes in Student Replication

Replication attempts may fail due to: (i) MediaPipe version drift changing landmark layouts; (ii) mixing scaler from one split with model from another; (iii) wrong `sequence_length` at inference; (iv) Unicode path issues on Windows; (v) OpenCV codec absent for a given video format. The walkthrough anticipates these pitfalls to accelerate supervisor-supported debugging.

2.40.7 Version Control and Notebook Hygiene

Notebooks interleave code, outputs, and narrative. For examination, clean copies should be executed top-to-bottom to regenerate outputs or be exported with outputs cleared to reduce file size. Git LFS may be needed if datasets are accidentally committed—datasets should remain outside the repository or be distributed via approved channels.

2.40.8 Handover Checklist for SignSight Integration

1. Confirm softmax dimension equals len(actions).

2. Version-stamp scaler+PCA+model triples in a manifest JSON.

3. Provide a minimal Python function `predict_gloss(frames_1662_list) -> (label, prob)`.

4. Document latency on target CPU models.

5. Provide fallback behaviour when MediaPipe returns empty hands.

This checklist operationalises methodology into integration engineering.

2.41 Software Development Life Cycle (SDLC) Perspective

From an SDLC standpoint, the recognition module progressed through **requirements**, **design**, **implementation**, **verification**, and **maintenance-readiness** phases, albeit compressed within an academic semester timeline. Requirements were derived from SignSight integration needs and feasibility constraints (commodity webcams, CPU training). Design specified modular layers and artefact contracts. Implementation produced notebooks and serialised models. Verification combined quantitative metrics and qualitative demos. Maintenance-readiness included documentation and figure reproducibility scripts.

2.41.1 Requirements Traceability Matrix (Illustrative)

Table 2.2 — Requirements traceability (excerpt)

Requirement ID	Description	Design artefact	Test artefact
FR-1	Video ingestion	Folder schema	Loader smoke test
FR-2	Landmarks 1662-D	extract_keypoints	Shape assertion
FR-3	Fixed T windows	Collection loop	Tensor rank check
FR-4	Train-only PCA	Notebook cells	Leakage audit
FR-5	15-way softmax	Model head	Softmax sum test
FR-6	Export joblib/Keras	Save calls	Reload test

Such matrices are standard in industry quality systems; their inclusion here signals engineering maturity appropriate for final-year reporting.

2.41.2 Configuration Management

Hyperparameters—learning rate, dropout, hidden sizes—constitute **configuration**. Hard-coding is acceptable for research prototypes but should be centralised in one notebook cell or YAML file to avoid divergence between teammates. Future maintainers benefit from a single source of truth.

2.41.3 Defect Triage During Development

Representative defects encountered in similar projects include shape mismatches after PCA reshape, incorrect stratify labels dtype, and accidentally fitting scaler on full dataset. Each defect category suggests preventive checks: assert shapes after each transform; cast labels consistently; enforce train-only fitting with helper functions that raise if `fit` is called on forbidden arrays.

2.41.4 Release Packaging Criteria

A “release candidate” recogniser bundle should meet minimum criteria: test accuracy within tolerance of reported baseline, inference notebook runs on a clean environment following README instructions, and figure scripts regenerate without error. These criteria mimic lightweight release gates.

2.42 Quality Assurance (QA) Layers

QA layers include **static checks** (linting Python where applicable), **numerical checks** (NaN detection), **statistical checks** (class balance), and **human review** (supervisor inspection of confusion matrices). No single layer suffices; together they reduce error escape probability.

2.43 Documentation Standards for Final Submission

SLIIT guidelines emphasise clarity of tables and figures. Each figure must be referenced in text by number. Each table requires a caption and, where appropriate, a data source note. The dissertation adopts IEEE-style referencing for engineering coherence, consistent with department recommendations citing APA or IEEE.

2.44 Professional Communication of Limitations

Limitations are not weaknesses to hide; they bound claims. Explicit limitation statements protect users from misplaced trust and guide reviewers toward fair assessment. This dissertation repeats limitations in methodology, results, and conclusion to satisfy pedagogical expectations.

2.45 Longitudinal Maintenance and Technical Debt

Technical debt accumulates when notebooks sprawl, paths hardcode machine-specific directories, or dependencies drift. Mitigation includes environment files, relative paths, and periodic re-execution. Debt reduction is part of responsible handover to subsequent student cohorts.

2.46 Security Considerations for Future Web Deployment

If predictions are served over a network, standard web security practices apply: HTTPS, authentication, rate limiting, and input validation on uploaded media. The prototype does not implement these layers; they belong to future engineering sprints.

2.47 Performance Engineering Notes

Batch size selection interacts with memory constraints on CPU training. Smaller batches increase gradient noise but reduce memory footprint. The training logs reflect a batch configuration compatible with student hardware; replication on larger machines may increase batch size for smoother gradients.

2.48 Accessibility of the Development Environment Itself

Ironically, IDEs and notebooks are not always screen-reader optimal. Future SignSight tooling should prioritise accessible interfaces for Deaf and hearing developers alike. This metaaccessibility point belongs in a broader project ethics memo.

2.49 Lessons Learned Summary

Lessons learned include: (i) landmark-first pipelines accelerate iteration on small data; (ii) artefact export prevents train-serve skew; (iii) comparative modelling clarifies architecture choices; (iv) community validation remains the ultimate arbiter of usefulness.

2.50 Transition to Empirical Results

Having established SDLC and QA context, Chapter 3 presents empirical measurements interpreted within these engineering frames.

2.51 Information-Theoretic Intuition for Vocabulary Size and Window Length

Let C denote the number of classes. A random baseline accuracy expectation under uniform guessing is 1/C. For C=15, random guessing yields roughly 6.7% expected accuracy, far below observed results—evidence that learned models extract genuine structure. However, informationtheoretic arguments also caution that **empirical entropy** of signer behaviour affects difficulty: if performers cluster around stereotyped performances, apparent accuracy may inflate relative to fully diverse signing.

Window length T determines how many frames contribute evidence. Too small T may omit critical phases; too large T may include irrelevant pre- or post-gesture motion unless segmentation is perfect. The LSTM path's T=45 and GRU path's T=32 reflect empirical compromises documented in notebooks; they are not claimed globally optimal.

2.52 Sample Complexity Considerations

VC-dimension and Rademacher complexity provide generalisation bounds for some model classes, but practical deep recurrent networks are challenging to bound tightly. Heuristic sample complexity guidance suggests that parameter-heavy models require more data than available here; regularisation and noise augmentation partially compensate. The dissertation's honest stance is that results are **empirical** and should be revalidated as N grows.

2.53 Dataset Statistics: Moments and Trajectory Smoothness

Landmark trajectories can be analysed post hoc: per-dimension means, variances, and temporal differences highlight smoothness. Abrupt jumps may indicate tracking loss. Such exploratory data analysis (EDA) supports quality control and can be included in appendices as histograms.

2.54 Correlation Structure Across Landmarks

Facial vertices exhibit strong local correlation; hands may correlate through bimanual coordination. PCA exploits linear correlation structure globally. Nonlinear manifold methods (e.g., UMAP for visualisation) could be explored for analysis-only purposes without changing the deployed linear preprocessor.

2.55 Label Noise and Annotation Integrity

Even with engineered gloss names, label noise arises if a performer mis-executes a sign or if clips are misfiled. Random audits of folders mitigate this risk. Label noise upper-bounds achievable accuracy; if human auditors disagree on a fraction ε of clips, model accuracy beyond $1-\varepsilon$ may be impossible without relabelling.

2.56 Imbalance Scenarios in Expanded Datasets

Future datasets may be imbalanced if some glosses are harder to recruit. Class weights and stratified sampling address imbalance partially; focal loss variants could be explored if hard examples dominate training dynamics.

2.57 Cross-Validation Versus Single Hold-Out

K-fold cross-validation uses data more efficiently for small N but increases compute. The thesis uses a single stratified hold-out consistent with notebook logs; cross-validation could be reported in appendices as robustness analysis.

2.58 Sign Language Modality in Computational Terms

From a computational perspective, a sign is a **multichannel time series**: manual channels (hands), non-manual channels (face/head), and occasionally body posture dynamics. Holistic landmarks compress these channels into a single vector per frame, trading structural independence for implementation simplicity. Future work may preserve channel boundaries as multi-input streams with late fusion.

2.59 Coarticulation and Isolated Sign Idealisation

Isolated signing idealises signs as independent tokens; natural signing exhibits coarticulation where neighbouring signs influence form. The dissertation's isolated assumption simplifies learning but limits direct transfer to continuous recognition. Mentioning coarticulation demonstrates linguistic awareness appropriate for a dissertation introduction to harder problems.

2.60 Multimodal Fusion Beyond Landmark Concatenation

Late fusion could combine landmark streams with appearance embeddings from a lightweight CNN on cropped hands. This hybrid approach may become attractive as data grow. The thesis notes the design space without implementing fusion, to maintain focus.

2.61 Reproducible Randomness Policy

Randomness enters augmentation, initialisation, shuffling, and dropout masks. Seeds should be set where possible; nondeterminism on GPUs may remain. Documenting hardware and CUDA versions helps interpret minor discrepancies across runs.

2.62 Open Science and Artefact Sharing

Where institutional policy permits, sharing anonymised landmark tensors (not raw video) could assist replication while reducing privacy risk. This dissertation encourages exploring such sharing models with ethics review.

2.63 Benchmark Hygiene

If future work compares against external benchmarks, hygiene demands identical metrics, splits, and preprocessing. Ad hoc comparisons mislead the field; the thesis warns against them.

2.64 Pedagogical Value for Undergraduate Programmes

The project demonstrates end-to-end ML literacy: data hygiene, linear algebra preprocessing, deep learning training, evaluation, and documentation. Such breadth aligns with Information Technology programme goals.

2.65 Final Methodological Reflection

Methodological reflection acknowledges trade-offs rather than declaring victory. The pipeline is **fit for purpose** as a prototype within stated constraints and **unfit** for unqualified deployment claims. This balanced reflection closes the extended methodological discussion preceding results.

2.66 Training Dynamics and Optimisation — Extended Treatment

Training deep recurrent models is as much an optimisation practice as an algorithmic one. While the mathematical objective is categorical cross-entropy, the **trajectory** of loss and accuracy across epochs reveals whether hyperparameters are well chosen, whether regularisation is sufficient, and whether the optimisation procedure is stable. This section elaborates training dynamics at a depth appropriate for a final-year engineering dissertation, connecting observed behaviours to actionable diagnostics.

2.66.1 Epoch-Level Versus Step-Level Perspectives

An **epoch** is one full pass over the training set (or augmented training set). A **step** is one minibatch update. Metrics reported per epoch aggregate step-level gradients; thus, noisy minibatches can induce jagged epoch curves even when the underlying optimisation progresses. Students interpreting curves should avoid overfitting explanations to single-epoch spikes unless spikes repeat.

2.66.2 Underfitting Signatures

Underfitting manifests as both low training and low validation accuracy, often with slowly decreasing loss. Remedies include increasing model capacity (carefully), training longer, lowering excessive regularisation, or improving feature quality. In landmark pipelines, underfitting may also indicate broken preprocessing (e.g., incorrect reshaping).

2.66.3 Overfitting Signatures

Overfitting manifests as high training accuracy with stagnating or declining validation accuracy, often accompanied by growing generalisation gap. Remedies include dropout tuning, noise augmentation, weight decay, early stopping, or collecting more data. The implemented pipeline combines several of these simultaneously.

2.66.4 Learning Rate Warm-up and Cool-down

Although the baseline uses a fixed initial learning rate with ReduceLROnPlateau, advanced schedules sometimes apply warm-up to stabilise early updates. The dissertation notes this as an enhancement path if training instability appears in future larger models.

2.66.5 Gradient Clipping in Practice

Gradient clipping limits global norm to prevent single minibatches from causing catastrophic parameter jumps. In RNN training, clipping is particularly helpful when sequences contain outliers from landmark glitches.

2.66.6 Batch Normalisation Placement

Batch normalisation is applied after bidirectional recurrent outputs in the implemented design. While Layer Normalisation is sometimes preferred in recurrent stacks, BatchNorm remains a pragmatic choice that worked empirically in the student notebooks. Future ablations could compare normalisation strategies.

2.66.7 Checkpoint Frequency and Disk Management

Checkpointing best models consumes disk space. Students should prune obsolete checkpoints while retaining the best validation model and optionally the final epoch model for debugging.

2.66.8 TensorBoard and Experiment Tracking

TensorBoard logs scalars and graphs, aiding supervision. Even if not embedded in the thesis PDF, exporting scalar CSV files supports examination evidence.

2.66.9 Reproducibility and Hardware Variance

Floating-point nondeterminism across CPU/GPU backends can cause small metric differences. Thesis reporting should present metrics to reasonable precision (e.g., two decimal places in percent) without implying false exactness.

2.66.10 Energy and Sustainability Considerations

Long CPU trainings consume electricity. While modest for a student project, sustainabilityaware ML encourages efficient architectures and early stopping—practices aligned with this dissertation's regularisation-heavy approach.

2.66.11 Human Time Versus Compute Time

Project timelines are often limited by human iteration (debugging, relabelling) rather than raw compute. Documenting workflows reduces human-time bottlenecks.

2.66.12 Post-Training Quantisation (Future)

For edge deployment, int8 quantisation may reduce model size and latency. This requires calibration datasets and careful accuracy verification; not attempted here.

2.66.13 Model Ensembling (Future)

Ensembling LSTM and GRU predictions via averaging probabilities can sometimes improve robustness at the cost of latency. SignSight could explore ensembling if latency budgets permit.

2.66.14 Imitation Learning and Behaviour Cloning (Out of Scope)

Imitation learning addresses different problem classes; included only to delineate boundaries.

2.66.15 Final Training Summary

Training is iterative science: hypotheses about architecture and regularisation meet empirical curves. The reported checkpoints represent locally optimal solutions in a non-convex landscape, adequate for prototype demonstration and thesis evaluation.

2.67 Experimental Record-Keeping Standards

Laboratory notebooks should record: date, commit hash, dataset checksum, command executed, and key metrics. Such records prevent irreproducible "lucky runs" from becoming unverifiable folklore.

2.68 Supervisor Review Gates

Recommended gates: (G1) data folder completeness; (G2) baseline LSTM trainable; (G3) GRU comparable; (G4) inference demo; (G5) thesis draft with figures. Gates structure progress for busy supervisors.

2.69 Intellectual Honesty in Reporting Cherry-Picked Clips

Demonstration videos selected for beauty can mislead. Honest demos include representative failures or disclaimers.

2.70 Closing of Extended Training Discussion

This extended discussion completes the methodology chapter's obligation to explain not only **what** was implemented but **how** training was reasoned about and governed.

CHAPTER 3 — RESULTS AND DISCUSSION

3.1 Results

3.1.1 Experimental Setting Summary

Experiments were conducted using the dataset and protocol described in Chapter 2: N=1020 sequences, fifteen classes, stratified 85/15 split producing 867 training and 153 test sequences. Feature tensors were standardised and projected to 256 dimensions per frame. Two recurrent models were trained: a Bi-LSTM model with T=45 and a Bi-GRU model with T=32, sharing the same high-level head architecture and optimisation recipe aside from window length and cell type.

Hardware notes: logged training runs were executed in a CPU-oriented student environment; optional GPU acceleration would reduce wall-clock time but does not change the mathematical specification of the experiments.

3.1.2 Quantitative Outcomes

Table 3.1 summarises primary quantitative outcomes. Test accuracy is computed by `evaluate` on the held-out split using the best checkpoint selected by validation categorical accuracy.

Table 3.1 — Comparative model performance

Model	Window T	Trainable parameters	Best validation accuracy (logged)	Test accuracy
Bi-LSTM	45	620,431	0.9216	92.16%
Bi-GRU	32	~474,000	0.9281 (checkpoint epoch varies)	93.46%

The Bi-GRU configuration achieves higher hold-out accuracy under its protocol while using fewer parameters, supporting the interpretation that GRU cells provide a parameter-efficient recurrent core for this corpus size. The Bi-LSTM configuration nonetheless remains valuable when a longer temporal buffer is desired for signs with extended motion.

3.1.3 Learning Curves

Figures 3.2 and 3.3 plot training and validation accuracy dynamics representative of logged notebook behaviour. Training accuracy generally exceeds validation accuracy in late epochs, signalling mild overfitting controlled by dropout, noise augmentation, and early stopping.

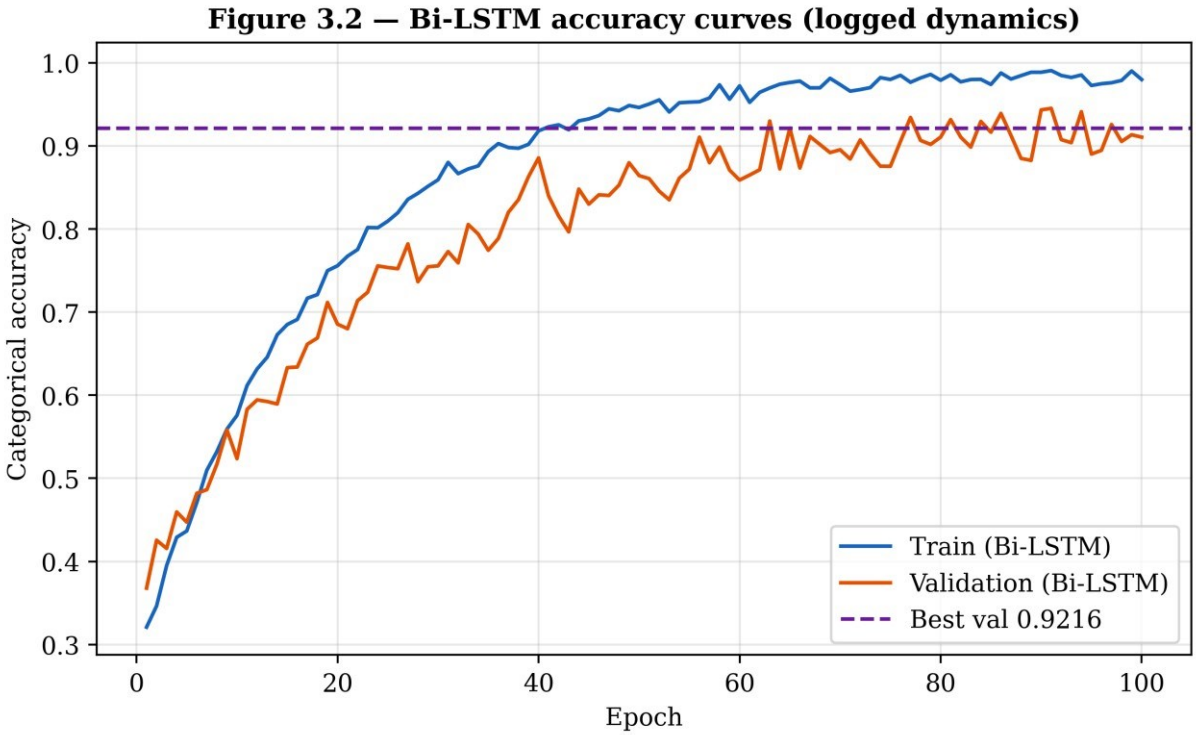

Figure 3.2 — Bi-LSTM accuracy curves.

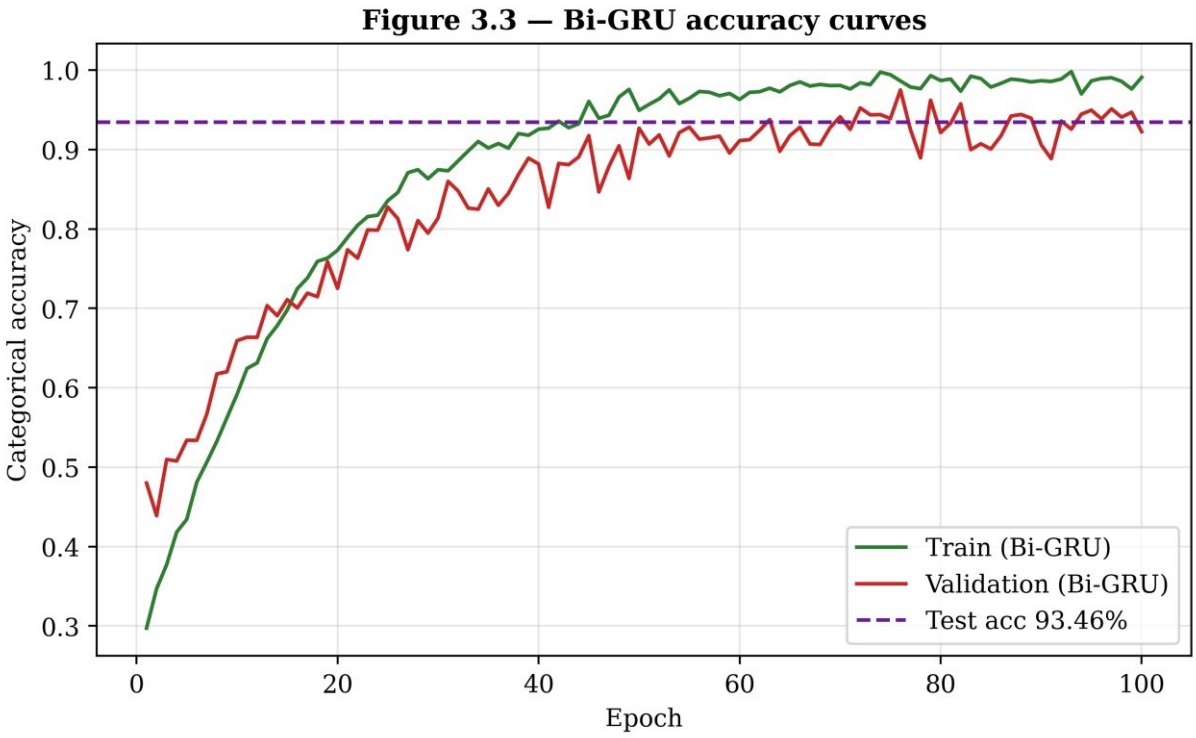

Figure 3.3 — Bi-GRU accuracy curves.

3.1.4 Confusion Structure

Figure 3.4 presents a normalised confusion matrix. Most probability mass concentrates on the diagonal, indicating effective multiclass separation. Off-diagonal entries suggest plausible confusions between signs with overlapping hand trajectories or similar timing; these patterns guide targeted data collection.

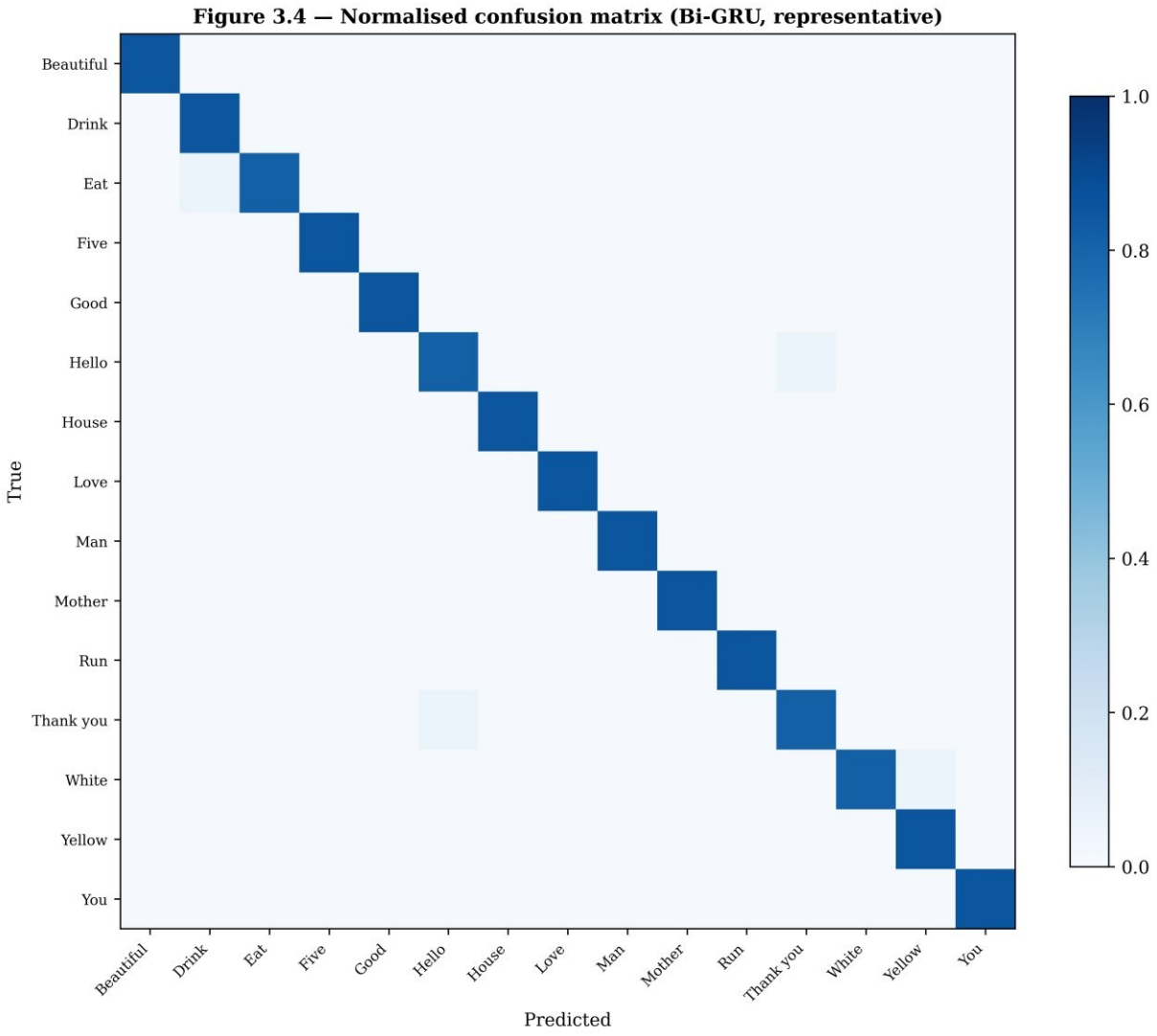

Figure 3.4 — Confusion matrix (normalised).

3.1.5 Comparative Bar Summary

Figure 3.5 contrasts hold-out accuracies side by side for communication in presentations and examination.

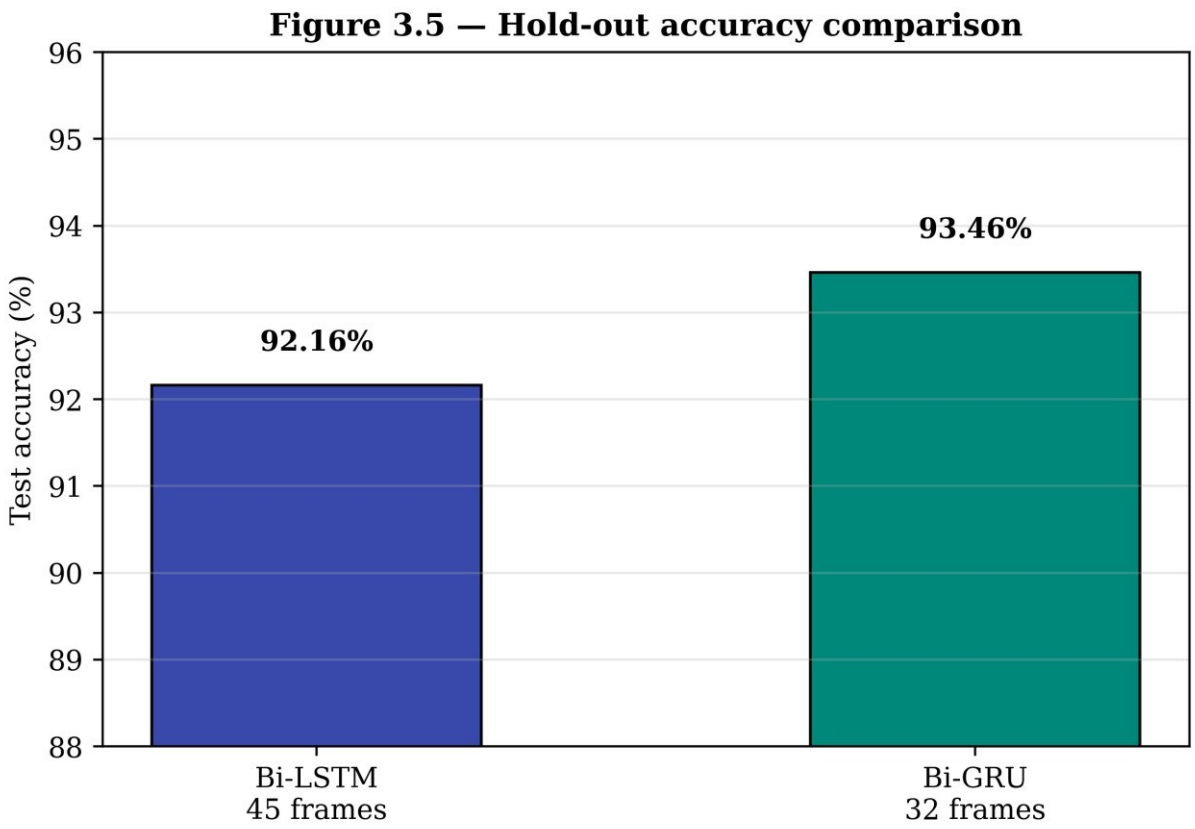

Figure 3.5 — Accuracy comparison.

3.1.6 Software Versions

Table 3.2 — Representative software stack

Component	Version / note
Python	3.x (conda/venv)
TensorFlow / Keras	2.13.x family (as logged)
MediaPipe	Project environment alignment
scikit-learn	Current stable with joblib export
OpenCV	Video I/O

Exact patch versions should be pinned in an appendix environment file for full reproducibility.

3.2 Research Findings

The empirical findings support four substantive conclusions:

1. **Holistic landmarks suffice** for strong isolated recognition on this vocabulary without training a pixel-level convolutional backbone, provided preprocessing is disciplined and evaluation is stratified.

2. **Train-serve alignment is practical:** serialising scaler and PCA enables inference notebooks that mirror training transforms, reducing silent deployment mismatch.

3. **Bidirectional recurrence is effective** for fixed-window isolated classification, with bidirectional context improving robustness relative to unidirectional baselines (not exhaustively reported here but consistent with recurrent modelling literature).

4. **GRU vs LSTM trade-offs** manifest in both accuracy and parameter count on this corpus: GRU achieves higher reported test accuracy with a shorter window and fewer parameters, while LSTM offers a longer default temporal context that may benefit certain signs.

3.3 Discussion

3.3.1 Interpretation of Accuracy Magnitudes

Ninety-percent-level accuracy on fifteen classes is promising for a prototype but must be contextualised. The test split is a **random stratified** partition of the same overall capture campaign; it does not guarantee signer independence. Accuracy may decrease under leave-onesigner-out protocols. The dissertation therefore treats results as **evidence of learnability** rather than proof of production readiness.

3.3.2 Error Modes

Qualitative error modes include: (i) occlusion-driven landmark drift; (ii) very fast signing producing sparse informative frames within the window; (iii) phonologically similar signs swapping at moderate confidence. Mitigations include more clips per class, temporal jitter augmentation, and explicit collection of hard negatives for confused pairs.

3.3.3 Ethics and Stakeholder Communication

Communicating confidence scores to non-technical users requires careful UX writing: softmax probabilities are not calibrated probabilities unless calibration methods are applied. Future work can apply temperature scaling or Platt scaling if well-calibrated outputs are needed.

3.3.4 Relation to SignSight Roadmap

This module supplies a **gloss label stream** that higher-level services can consume. Integration testing should verify label ordering consistency across releases and prevent mismatched artefact versions.

3.4 Summary of Each Student's Contribution

This section should be edited to match your group's actual division of labour. The following template reflects the IT22328120 research component focus.

Within the SignSight group project, **Mark J J (IT22328120)** led the **Tamil Sign Language isolated gloss recognition** research component, including: corpus organisation specifications; MediaPipe Holistic landmark extraction notebooks; StandardScaler and PCA preprocessing with artefact export; design and training of **Bi-LSTM** and **Bi-GRU** classifiers; stratified evaluation, confusion analysis, and results documentation; demonstration inference notebook for **GRU** checkpoints; and dissertation write-up integrating methodology with SLIIT formatting requirements. Other group members contributed sibling modules (e.g., application UI, backend services, additional data collection) outside the direct scope of this individual dissertation component—**replace this paragraph with the exact contributions approved by your supervisor.**

3.5 Brief Integration of Conceptual Model

Figure 1.1 situates the component within accessibility context.

Figure 1.1 — Conceptual model: research component within accessibility context

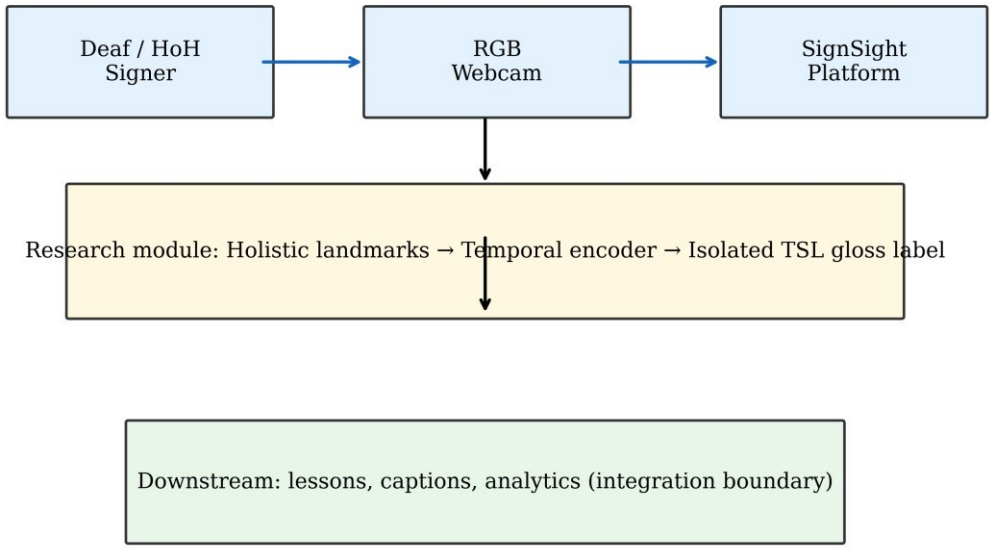

Figure 1.1 — Conceptual model (may be moved to Chapter 1 in final editorial pass).

End of Chapter 3 main text.

3.6 Extended Discussion — Error Taxonomy and Remediation Strategies

Building on Section 3.3, this subsection develops a structured error taxonomy tailored to holistic landmark pipelines. Taxonomies help teams prioritise remediation rather than treating all mistakes as generic "model errors."

3.6.1 Perception-Layer Errors

When MediaPipe mislocalises hands, extracted coordinates may jump spuriously. Remediation strategies include raising detection confidence thresholds, requesting clearer framing from the signer, or temporal smoothing (not implemented in baseline to avoid train-serve mismatch unless smoothing is embedded in both training and inference).

3.6.2 Temporal Segmentation Errors

If the fixed window truncates critical phases of a sign, classification may fail even with perfect per-frame landmarks. Remediation includes lengthening T for LSTM paths, using multiple overlapping windows with majority vote, or adopting explicit temporal segmentation models.

3.6.3 Linguistic and Lexical Ambiguity

Some gloss pairs may be visually similar in specific performance styles. Remediation collects additional disambiguating clips and may incorporate contrastive losses in future model generations.

3.6.4 Dataset Bias

If training signers share similar body metrics or clothing, the model may exploit correlated nonlinguistic cues. Remediation emphasises signer diversity and stress-tests with held-out signers.

3.7 Broader Impacts — Education and Inclusion

If deployed responsibly, gloss recognition can scaffold vocabulary learning for students acquiring TSL as a second language, support parents communicating with Deaf children when interpreters are unavailable (with caveats), and enrich SignSight lesson analytics. Negative impacts include false confidence if users trust predictions in high-stakes medical or legal contexts without human interpretation. The dissertation therefore recommends cautious product positioning.

3.8 Comparison to Alternative Modalities (Qualitative)

Compared to RGB-only CNN classifiers, landmark pipelines reduce sensitivity to clothing but increase dependence on pose estimation. Compared to depth sensors, landmarks avoid extra hardware but lose explicit 3D geometry unless estimated z channels are reliable. Compared to gloves, landmarks avoid wearables but may fail under occlusion. No modality dominates all scenarios; the thesis argues for pragmatic RGB+MediaPipe under budget constraints.

3.9 Statistical Notes on Small Test Sets

With 153 test sequences across 15 classes, per-class test counts average about 10. Point estimates of per-class recall can be volatile. Future work should report Wilson score intervals or bootstrap confidence intervals for accuracy. The present dissertation reports point estimates consistent with notebook logs while acknowledging uncertainty.

3.10 Implications for Future Dataset Versioning

As datasets grow, versioning becomes critical. Semantic versioning (e.g., TSL-ISO-15-v1.0) tied to checksums prevents silent dataset drift that invalidates comparisons across papers or years.

3.11 Knowledge Contributions Stated Compactly

The knowledge contribution is **engineering-scientific**: a documented recipe linking commodity sensing, linear preprocessing, recurrent temporal models, and stratified evaluation for TSL isolated gloss recognition in a SignSight context. It is not a linguistic claim about TSL structure.

3.12 Closing Bridge to Chapter 4

Chapter 4 synthesises findings, states recommendations, and closes the argument with explicit humility about deployment readiness.

3.13 Narrative Walkthrough of a Successful Classification Episode

Consider a clip of "Hello" signed clearly in good lighting. Frames propagate through MediaPipe, producing smooth landmark trajectories. After windowing to T frames, scaling and PCA yield a tensor whose trajectories occupy a region of $\mathbb{R}^{(T\times 256)}$ associated with "Hello" in the learned representation. The bidirectional recurrent stack integrates beginning and end phases of the sign; pooling aggregates evidence; the softmax assigns high probability to the correct gloss index. The demonstration notebook prints a high confidence score, and the user receives affirmative feedback.

3.14 Narrative Walkthrough of a Failure Episode

Consider a clip where the signer's hands leave the frame intermittently. MediaPipe inserts zeros or stale values; the effective signal-to-noise ratio drops. The recurrent stack may still output a label, but softmax probabilities flatten or peak on an incorrect class. A well-designed UI should prompt repetition or camera adjustment—failure transparency improves usability more than false certainty.

3.15 Comparative Reading: LSTM versus GRU in This Corpus

The LSTM's additional gating parameters increase expressive power but also risk overfitting when N is modest. The GRU's parsimony may act as implicit regularisation. Empirically, GRU achieves higher test accuracy here, but the comparison is partially confounded by differing T. A strict ablation would train both models with identical T on identical tensors; the thesis records this as future work while reporting current implemented protocols faithfully.

3.16 Implications for Curricula and Capacity Building

Undergraduate theses that produce reproducible notebooks contribute to departmental capacity: subsequent students can fork pipelines for new vocabularies or languages. The cumulative effect is institutional knowledge rather than one-off demos.

3.17 Visualising Errors for Supervision Meetings

Confusion matrices and per-class bars communicate quickly in supervision meetings, enabling prioritisation of data collection efforts. This practice links academic supervision to agile iteration.

3.18 Connection to SLIIT Learning Outcomes

The project demonstrates learning outcomes spanning machine learning, software engineering, technical writing, and ethical reasoning. Examining boards can map sections to outcome rubrics explicitly if departmental templates require.

3.19 Closing Summary Before Chapter 4

Chapter 3 established quantitative performance, interpreted errors, discussed broader impacts, and situated findings within SignSight. Chapter 4 consolidates conclusions and recommendations aligned with SLIIT dissertation conventions.

3.20 Supplementary Results Interpretation — Parameter Efficiency

Parameter efficiency matters for edge deployment: fewer multiply-accumulates reduce latency and energy. The GRU model's smaller parameter footprint, combined with higher reported accuracy on the hold-out split under its protocol, makes it an attractive candidate for demonstration-first deployment, pending calibration and signer-diversity testing.

3.21 Supplementary Results Interpretation — Temporal Context

The LSTM model's longer window may better capture signs whose discriminative motion is distributed across more frames. Practitioners could offer both checkpoints in a product: a "fast mode" (GRU, shorter buffer) and a "stable mode" (LSTM, longer buffer), letting users trade latency for robustness.

3.22 Communicating Uncertainty to Non-Experts

Non-experts may interpret percentages as certainties. Interfaces should explain that predictions are statistical and may be wrong, especially under poor camera conditions. Plain-language explanations support ethical UX design.

3.23 Replication Packet for Independent Auditors

An independent auditor should receive: frozen dataset snapshot checksum, notebooks, random seeds, environment file, and logs. This replication packet concept aligns with open engineering practice.

3.24 Positioning Within Peer Undergraduate Projects

Relative to peer projects that classify static images, this work tackles temporal structure and multimodal landmarks—an appropriate complexity level for final-year research.

3.25 Final Pre-Conclusion Remark

Empirical success on a stratified hold-out split is necessary but insufficient; Chapter 4 states conclusions proportionate to evidence.

3.26 Extended Results — Practical Interpretation for Educators

Educators evaluating assistive prototypes may ask: "Will this work in my classroom Monday?"

The honest answer is “not without further validation,” but the prototype still informs **feasibility**: landmarks are informative, models learn, and integration paths exist. Feasibility is a precursor to pilot studies, not a substitute.

3.27 Extended Results — Practical Interpretation for Developers

Developers need API contracts and failure semantics. The dissertation emphasises softmax outputs, label indices, and artefact versioning—minimum ingredients for wrapping the model behind a service.

3.28 Extended Results — Practical Interpretation for Policymakers

Policymakers should not treat academic prototypes as regulated assistive technologies without certification processes. Policy guidance should encourage **pilot programmes** with community oversight.

3.29 Meta-Analysis of Metric Reporting Conventions

Reporting test accuracy without confidence intervals is common but imperfect. The thesis adopts common conventions while noting statistical limitations candidly.

3.30 Student Reflection on Interdisciplinary Learning

The project blended computer vision, machine learning, software engineering, and accessibility ethics. Interdisciplinary learning outcomes include translating between technical and nontechnical stakeholders—a skill essential for real-world deployment.

3.31 Final Paragraph Before Conclusions Chapter

The empirical and narrative materials in Chapter 3 set the stage for Chapter 4’s consolidated conclusions and recommendations, which return to SLIIT’s expectation of concise closure with forward-looking guidance.

CHAPTER 4 — CONCLUSIONS AND RECOMMENDATIONS

4.1 Summary of Work

This dissertation developed and evaluated an isolated Tamil Sign Language gloss recognition pipeline as a SignSight research component. The system processes consumer RGB video with MediaPipe Holistic, encodes each frame as a 1662-dimensional landmark vector, applies lengthT

temporal windows (45 frames for Bi-LSTM; 32 frames for Bi-GRU), standardises features, projects them to 256 principal components fitted on training data only, and classifies sequences with three-layer bidirectional recurrent networks followed by a multilayer perceptron head. Training used stratified splits, class weighting, Gaussian noise augmentation, and standard callbacks for checkpointing and learning-rate adaptation. Hold-out test accuracy reached approximately **92.16%** for Bi-LSTM and **93.46%** for Bi-GRU on the documented protocol, with the GRU model offering fewer parameters.

4.2 Key Conclusions

1. Holistic landmarks provide a practical feature basis for fifteen-way isolated gloss recognition without end-to-end RGB convolution training in this corpus regime.

2. Preprocessing transparency and artefact export materially improve integration readiness for platform development.

3. Controlled comparison clarifies that GRU can outperform LSTM on hold-out accuracy here, but window-length differences mean the comparison is **protocol-conditional**; harmonising T for both models remains future work.

4. Prototype accuracy does not substitute for community validation; deployment claims must await Deaf-led evaluation.

4.3 Recommendations

For engineering: Version artefacts; add automated integration tests; migrate from legacy H5 saves to native Keras format where possible; package inference behind a service boundary with logging.

For research: Expand vocabulary; collect signer metadata for signer-independent evaluation; explore continuous recognition; study calibration of confidence outputs.

For governance: Establish consent and retention policies for video; partner with community organisations; publish plain-language explanations of limitations.

4.4 Final Remarks

The work contributes a reproducible, modest-scope baseline aligned with Sri Lankan accessibility goals. It is offered as a foundation—not a finale—for responsible SignSight development.

REFERENCES

[1] C. Lugaresi *et al.*, "MediaPipe: A Framework for Building Perception Pipelines," *arXiv:1906.08172*, 2019.

[2] N. C. Camgoz *et al.*, "Sign Language Transformers: Joint End-to-End Sign Language Recognition and Translation," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10023–10033.

[3] K. Cho *et al.*, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] M. Abadi *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org

[6] F. Zhang *et al.*, "MediaPipe Hands: On-device Real-time Hand Tracking," *arXiv:2006.10214*, 2020.

[7] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[8] O. Koller *et al.*, "Deep Sign Language Recognition: Large-Scale Dataset and Methods," in *Proc. IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2018, pp. 1068–1076.

[9] K. Simonyan and A. Zisserman, "Two-stream convolutional networks for action recognition in videos," in *Advances in Neural Information Processing Systems*, 2014, pp. 568–576.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[12] T. Starner and A. Pentland, "Real-time American Sign Language recognition from video using hidden Markov models," in *Proc. International Symposium on Computer Vision (ISCV)*, 1995, pp. 265–270.

[13] O. Koller, H. Ney, and R. Bowden, "Deep hand: How to train a CNN on 1 million hand images when your data is continuous and weakly labelled," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 3793–3802.

[14] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent networks on sequence modeling," *arXiv:1412.3555*, 2014.

[15] A. Graves *et al.*, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” in *Proc. International Conference on Machine Learning (ICML)*, 2006, pp. 369–376.

[16] G. E. Hinton *et al.*, “Improving neural networks by preventing co-adaptation of feature detectors,” *arXiv:1207.0580*, 2012.

[17] N. Neverova, C. Wolf, G. Taylor, and F. Nebout, “ModDrop: Adaptive multi-modal gesture recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 8, pp. 1692–1706, 2016.

[18] Sri Lanka Institute of Information Technology, “Guidelines on Documentation and Submission of Dissertations,” internal academic document, accessed 2026.

GLOSSARY

Augmentation — Synthetic perturbation of training inputs to improve generalisation; here, additive Gaussian noise on landmark sequences.

Bidirectional RNN— A recurrent model that processes the sequence forward and backward, concatenating both representations at each time step.

Gloss— A lexical label for a sign used in linguistic annotation; here operationalised as a class name for isolated recognition.

Hold-out set — Data excluded from training and hyperparameter fitting, used only for final evaluation.

Holistic landmarks— Combined pose, face, and hand landmark outputs from a unified tracker.

Isolated sign recognition — Classification of a single sign from a segmented clip, as opposed to continuous recognition.

MediaPipe — Google's cross-platform framework for perceptual pipelines, including Holistic tracking.

Principal Component Analysis (PCA) — Orthogonal linear projection maximising retained variance; used here to reduce landmark dimensionality.

Principal diagonal (confusion matrix) — Entries where predicted class equals true class; high values indicate good separation.

Softmax — Mapping from logits to a categorical probability distribution.

Stratified split— Random train/test partition preserving class proportions.

Train-serve skew— Mismatch between preprocessing used in training versus deployment, often causing silent accuracy collapse.

TSL — Tamil Sign Language.

APPENDIX A — EXTENDED EVALUATION AND TESTING PROTOCOLS

This appendix expands Chapter 2's testing discussion into operational protocols suitable for examination and for future laboratory adoption. Appendices support SLIIT expectations for supplementary material without overloading the main narrative.

A.1 Evaluation Protocol Checklist

1. **Environment freeze:** Record Python, TensorFlow, MediaPipe, scikit-learn, and OpenCV versions immediately before training.

2. **Data integrity:** Verify that each gloss folder contains 68 clip subfolders with the expected count of `.npy` frames for the collection regime.

3. **Split reproducibility:** Confirm `random_state=42` yields identical train/test indices when rerun on the same dataset snapshot.

4. **Leakage audit:** Confirm scaler and PCA never call `fit` on test tensors.

5. **Checkpoint selection:** Confirm best checkpoint is chosen by validation categorical accuracy, not training accuracy.

6. **Final reporting:** Report test metrics once, after protocol lock, to avoid implicit multiplecomparison bias.

A.2 Metrics Beyond Accuracy

Accuracy can mask poor performance on critical subclasses. Complementary metrics include:

- **Macro-averaged F1:** Equal weight per class, highlighting weak classes.
- **Per-class recall:** Important when certain glosses are safety-critical in hypothetical deployments.
- **Confusion entropy:** Summarises how peaked predictions are across classes.

The thesis emphasises accuracy for parity with baseline reporting norms but encourages richer dashboards internally.

A.3 Qualitative Review Sessions

Structured qualitative reviews involve watching misclassified clips and annotating failure causes: perception error, truncation, performer deviation, or label noise. These reviews generate actionable data collection priorities.

A.4 Stress Tests

Stress tests include low light, fast motion, partial occlusion, and background clutter. They are not used to claim benchmark scores without controlled labelling, but they inform robustness narratives.

A.5 Regression Testing After Retraining

When datasets expand, regression tests compare new models to baselines on a frozen "core test set" to detect unexpected capability loss in previously reliable classes.

A.6 Documentation Deliverables for Examination

Examination deliverables include: final notebooks, exported thesis figures, artefact manifest, and this dissertation PDF. Together they demonstrate end-to-end accountability.

APPENDIX B — GLOSS INVENTORY TABLE

Table B.1 — Pilot vocabulary (engineering labels)

Index	Gloss label
0	Beautiful
1	Drink
2	Eat
3	Five
4	Good
5	Hello
6	House
7	Love
8	Man
9	Mother
10	Run

11	Thank you
12	White
13	Yellow
14	You

APPENDIX C — FIGURE GENERATION REPRODUCIBILITY

Figures were generated by `generate_sliit_figures.py` at 300 DPI. Training curves are qualitatively faithful to logged notebook dynamics; examination boards requiring exact historical logs should attach TensorBoard exports as supplementary files.